



Automating on-site object inspection with a quadruped robot and BIM

Jaakko Tuomisto ^a, Yuan Zheng ^b, Inshu Chauhan ^b, Olli Seppänen ^b, Joni Pajarinen ^a

^a Aalto University, School of Electrical Engineering, Department of Electrical Engineering and Automation, Maarintie 8, Espoo, 02150, Uusimaa, Finland

^b Aalto University, School of Engineering, Department of Civil Engineering, Rakentajanaukio 4, Espoo, 02150, Uusimaa, Finland

ARTICLE INFO

Keywords:

Autonomous mobile robots
Adaptive navigation
Object inspection
Construction progress monitoring
BIM
Next-best-view planning
Vision-language models

ABSTRACT

Deploying robots into construction sites and built environments using Building Information Modeling (BIM) is difficult due to environmental changes and inconsistencies between BIM and real-world structures. To overcome these challenges, an integrated framework is proposed for automated inspection featuring adaptive BIM-driven navigation for a quadruped robot. The framework addresses challenges of localization, mapping, and navigation in challenging environments. By leveraging BIM for initialization and planning, it dynamically verifies BIM elements through sensor data, updating the map to distinguish between fixed structures, missing elements, and dynamic obstacles. This enables robust navigation despite environmental uncertainties. The capability supports automated tasks across the building lifecycle, demonstrated through object inspection using adaptive next-best-view (NBV) planning and vision-language model (VLM) analysis without task-specific training. Both simulated experiments and real-world validation confirm strong performance despite dynamic obstacles and BIM discrepancies. The approach enhances robotic adaptability in construction and facility management contexts while reducing manual inspection requirements.

1. Introduction

Gathering accurate and timely information about building objects is a critical task throughout the lifecycle of a built environment, from construction to operation and maintenance [1,2]. This targeted acquisition of object-level data, often called object inspection, supports tasks such as construction progress monitoring [3] and quality assurance [4]. Post-construction, it supports facility management [5] by verifying the presence, placement, and condition of building elements and equipment. Traditionally, these tasks rely on manual walkthroughs, which are widely considered time-consuming and labor-intensive, resulting in inconsistencies and insufficient data coverage [6].

To address these limitations, research has shifted toward automated object-level data collection [1]. Various technologies have been explored for this purpose, including Radio Frequency Identification (RFID) [7], Bluetooth low energy (BLE) [8], Light Detection and Ranging (LiDAR) [6,9], photogrammetry [10], and computer vision [11]. Among these, LiDAR and vision-based methods stand out as particularly effective for continuous or on-demand inspection [12].

However, deploying stationary sensors faces significant challenges. Stationary sensors relying on line-of-sight must be carefully positioned to cover relevant areas and maintain visibility of target objects, especially on active construction sites [13]. While networks of multiple sensors improve coverage [14], they are often costly and difficult to

manage or reposition. Even wearable sensors, such as helmet-mounted cameras, still rely on manual walkthroughs. This lack of flexibility, coupled with the fact that certain inspection areas may be hazardous or difficult to access, highlights the need for autonomous mobile solutions.

To meet this need, research has begun to explore mobile robotic solutions that can autonomously navigate construction or facility environments and collect data with reduced human involvement. While Unmanned Aerial Vehicles (UAVs) have shown promise in outdoor scenarios [15], Unmanned Ground Vehicles (UGVs) are better suited for indoor environments due to safety considerations [16]. These autonomous robots can navigate between multiple viewpoints, offering a potential solution that addresses the limitations of fixed sensors. Their capabilities are further enhanced by Building Information Modeling (BIM), which provides a priori geometric and semantic information to aid in localization, navigation, and task planning [17–19].

Despite this potential, the practical deployment of BIM-driven robots is hindered by several challenges. A robot must reliably localize itself relative to the BIM coordinate frame, seamlessly integrate prior BIM information with real-time sensor data for mapping, and navigate discrepancies between as-designed BIM and as-built reality. In practice, this means handling missing or partially built structures. Adding to this complexity, robots must contend with moving personnel and dynamic obstacles such as construction materials and equipment. Successfully

* Corresponding author.

E-mail address: jaakko.tuomisto@aalto.fi (J. Tuomisto).

and simultaneously addressing these challenges within an integrated system remains a barrier to effective real-world operation. *This raises the question: how can robots synthesize structural BIM information with semantic understanding and real-time sensor data to complete inspection tasks in such challenging conditions?*

Although significant research has addressed these components individually, such as techniques for BIM-based localization [20,21], mapping [17,22], and navigation planning [23–26], few systems effectively integrate them to handle the combined challenges. Specifically, many existing approaches display limited adaptability. For instance, stop-and-go mapping systems [27,28] optimize predetermined scan positions but lack mechanisms to adjust these positions when encountering unexpected obstacles or other structural changes. Similarly, object inspection systems [16,29] rely on precomputed viewpoints based on BIM without strategies for handling occlusions or model differences. Third, even when BIM-driven navigation is used [23–26], systems often lack adaptations for in-progress structures or other dynamic obstacles, both of which are common in construction and renovation contexts. This reveals a key insight: treating BIM as a fixed blueprint to be followed prevents robots from adapting to real-world conditions. Instead, effective autonomous operation requires treating BIM as probabilistic prior knowledge that must be continuously validated and refined through sensor observations. Consequently, a gap exists for practical integrated robotic systems that can reliably execute tasks while simultaneously handling localization, mapping, navigation, and environmental dynamics.

To bridge this gap, this paper proposes an integrated framework for adaptive navigation and mapping for autonomous robots operating in dynamic environments. Our approach synthesizes different information types for specific purposes: BIM structures validated against sensor data define navigation constraints, sensor-detected unmodeled obstacles trigger obstacle avoidance, and BIM metadata enables semantic object verification through vision-language models. We demonstrate a complete system for automated on-site object inspection using a quadruped robot. We specifically target structured environments where the system can leverage architectural elements defined in BIM as a source of prior information while continuously verifying and updating the robot's environmental understanding with sensor data to increase the robustness of navigation and task execution.

The main contributions of our work are:

1. **BIM-Driven Localization:** A practical localization method that leverages 3D LiDAR SLAM initialized with BIM geometry. This approach provides consistent alignment between sensor observations and the BIM coordinate frame throughout operation, enabling the environmental mapping and navigation capabilities that follow. The approach shows robustness to dynamic obstacles, floor-level clutter, and missing BIM structures.
2. **BIM-Driven Mapping and Navigation:** A navigation system that integrates BIM with LiDAR data. It uses BIM as prior information to inform path planning, avoiding costly exploration, while dynamically distinguishing between static structural elements, such as walls, and dynamic obstacles, such as construction materials. BIM structures verified with LiDAR observations are treated as hard constraints to avoid revisiting known areas and to ensure collision-free paths, while those confirmed missing (e.g., incomplete walls) are removed from the map. Temporary obstacles detected by sensors function as soft constraints. Our semantic approach reduces map clutter, enabling the robot to avoid static structural elements while adapting to dynamic conditions.
3. **Next-Best-View Planning:** An adaptive next-best-view planner that leverages our dynamically updated 3D map. This enables effective viewpoint selection for object inspection, even when occlusions or structural changes make viewpoints based solely on BIM ineffective.

4. **Vision-Language Model Integration:** The integration of a foundational vision-language model (VLM) to automatically classify the installation status of target objects from images, using textual BIM metadata, without requiring object-specific training data.
5. **End-to-End Validation:** Validation of the integrated framework through both realistic simulations that model sensor data and dynamic changes in the environment, and a practical test in a real-world environment.

This research advances the field by introducing an integrated robotic system for object inspection, supported by an adaptive BIM-driven navigation framework. Collectively, these contributions provide a foundation for automating a wider range of robotic tasks on dynamic construction sites or buildings.

2. Background

This section discusses related research in construction site monitoring, robotics, BIM integration, and data analysis.

2.1. Data acquisition in construction and built environments

Efforts to overcome the administrative overhead and data deficiencies of traditional manual methods [13,30–32] have led to two main automated approaches. The first involves attaching tracking devices like Radio Frequency Identification (RFID), Ultra-Wideband (UWB), or Bluetooth Low Energy (BLE) to building components [8,33–35]. While effective for locating materials, these tag-based methods cannot verify the proper installation or condition of components, limiting their utility for detailed inspection tasks.

The second, more common approach uses environmental sensors. Cameras combined with photogrammetry can produce rich 3D data but require extensive, overlapping imagery [3,10]. In contrast, LiDAR and depth cameras directly capture 3D point clouds, with LiDAR offering superior accuracy at a higher cost and operational complexity [36–38].

However, the effectiveness of these sensors is fundamentally tied to their deployment. Fixed sensors are hindered by line-of-sight constraints, necessitating either costly multi-sensor networks [14] or manual data collection with human-carried devices like 360 cameras [39]. Compounding this physical deployment challenge are the issues of data handling. The vast datasets generated require significant processing with computer vision or point cloud analysis techniques to extract relevant information [11,40,41]. Ultimately, this information must be contextualized against the design intent, typically contained within a Building Information Model (BIM) [4], a research area that has seen extensive exploration [17,19,27,42]. Therefore, effective solutions must not only address the challenge of sensor placement but also integrate data processing and BIM context awareness.

2.2. Robotic monitoring and inspection

A predominant approach in robotic monitoring in construction applications is “stop-and-go mapping”, where robots navigate to a series of predetermined viewpoints to capture sensor data [28,42]. These systems often leverage BIM *a priori* to optimize sensor positions for coverage. For instance, Park et al. [27] use this strategy with a Boston Dynamics Spot robot navigating a pre-mapped environment via the GraphNav system. This approach inherently relies on a static, pre-planned mission that does not adapt to on-site changes.

Beyond general site scanning, other research focuses on targeted object inspection. Muhammad et al. [16] propose the Robot-Assisted Object Detection (RAOD) framework, which uses a 2D environmental representation for navigation and plans viewpoints that require initial target visibility for fine-tuning maneuvers. Other systems focus on verifying component installation by comparing sensor data to BIM-derived checklists. A representative work by Liao et al. [29] uses this method to compare elements detected via YOLOv5 and an RGB-D sensor against a BIM plan. This strategy presumes a strong correspondence between the as-designed model and the as-built site.

2.3. BIM-Driven Localization, Mapping, and Navigation

Building Information Modeling (BIM) provides geometric and semantic information about planned structures, which can be used by robots for self-localization, initializing maps, and efficient path planning without extensive exploration. This section examines prior approaches to leveraging BIM for these applications, highlighting both advances and limitations.

2.3.1. BIM for robot localization

Accurate localization is essential for autonomous mobile robots to navigate and execute tasks relative to a building's structure. While onboard sensors like wheel encoders and Inertial Measurement Units (IMUs) can track motion, they suffer from cumulative drift, which degrades positional accuracy over time. This makes it difficult to maintain a precise position relative to BIM, especially since manually setting an accurate initial position is also prone to error. To counteract drift, robots use environmental referencing, typically through *Simultaneous Localization and Mapping (SLAM)*. SLAM anchors position estimates to observable features using sensor modalities like cameras for *Visual SLAM (VSLAM)* or laser scanners for *LiDAR SLAM*.

While SLAM compensates for drift, integrating BIM data allows robots to localize within the building's global coordinate system. The integration approach often depends on the dimension of the BIM information used. A prominent method is to use 2D LiDAR with maps derived from BIM. For example, Liao et al. [29] propose converting BIM geometry into a binary, rasterized 2D map for LiDAR-based Adaptive Monte Carlo Localization (AMCL). However, such methods face two key challenges in practice: ground-level clutter can occlude structural features from the single-plane sensor, and static maps from as-designed models do not account for potential discrepancies.

Expanding on BIM-initialized 2D approaches, Torres et al. [43] compare graph-based and particle filter methods, specifically AMCL and General Monte Carlo Localization (GMCL) using BIM-derived 2D occupancy grids, finding that GMCL excels at global localization while graph-based algorithms are better for pose tracking. A common technique in these methods is to filter the BIM to extract only permanent structures like walls, excluding dynamic or invisible elements like doors and windows [17,43,44].

Moving to 3D, approaches vary in how they integrate BIM. Moura et al. [17] introduce a method that initializes 3D SLAM with BIM. They convert BIM data into a 2D occupancy grid for simulating robot movement and generating LiDAR point clouds using ray tracing as if the robot were driving around the site. This simulated LiDAR data is then inserted into the pose graph of Google's Cartographer SLAM system. In contrast, Schaub et al. [45] first build a 3D point cloud using SLAM and only then register it with a BIM-generated point cloud via axis alignment, template matching, and ICP refinement. A key limitation of the latter method is that the BIM model does not actively assist or correct the SLAM process itself.

To improve 3D registration, Yin et al. introduce a semantic approach [21]. The process first performs an initial ICP alignment, then refines it by labeling points with BIM semantic categories (e.g., Ceilings, Floors, Walls) and applying a weighted ICP. While effective, this highlights the recurring challenge that localization accuracy is diminished by discrepancies between as-planned and as-built conditions [46]. Furthermore, the authors acknowledge their method's own limitations: its sequential process creates a critical dependency on the success of the initial, non-semantic ICP alignment, and it assumes a static environment not suitable for dynamic settings.

While LiDAR is prevalent, vision-based methods offer an alternative. Shan et al. [47] integrate monocular SLAM by detecting line segments, determining their 3D positions with ORB features and triangulation, and matching them to BIM-derived lines. Similarly, Zhao et al. [19] use a CNN (YOLOv3) to detect architectural features like doors for matching. This position is refined using 2D AMCL, which, as Torres

et al. note [43], can be a limitation compared to more robust graph-based methods. Both visual approaches are susceptible to poor lighting, feature-sparse environments, and the difficulty of relating visual cues to BIM geometry.

A recurring theme across localization approaches is establishing a robust correspondence between sensor data and BIM geometry to enable reliable localization. This influences both the selection of the SLAM approach and the employed sensors. While visual SLAM can be robust in many scenarios, it displays clear limitations for BIM-relative positioning. Visual features typically lack a direct geometric counterpart in the BIM, making reliable alignment difficult. Furthermore, a building's visual appearance changes significantly over its lifecycle, invalidating these features over time. These limitations explain the predominant use of LiDAR, as it directly measures the permanent geometric structures, such as walls, floors, and ceilings, that are both explicitly represented in BIM and remain physically stable.

2.3.2. BIM for environmental mapping

Mobile robots depend on accurate environmental maps that represent both permanent structures and temporary obstacles. As with localization, BIM can provide valuable prior information about planned structures, but integrating it with real-time sensor data is a challenge. However, while localization systems typically involve or generate maps as part of their operation, they are often optimized for pose estimation, limiting their utility for other applications [48].

To facilitate tasks such as viewpoint planning and BIM-aware navigation, maps could benefit from semantic information that distinguishes between as-designed and as-built data, and from compatibility with common robotics frameworks (e.g., ROS2 Nav2). To support navigation without preparatory SLAM mapping, maps can be initialized with BIM geometry [27,28]. For inspection tasks, including 3D information could facilitate more accurate viewpoint planning: unlike 2D occupancy grids, 3D maps capture occlusions at various heights, enabling the detection of unoccluded viewpoints for inspection targets even in the presence of difficult obstacles.

Autonomous mobile robots typically use occupancy grids as maps, where each cell represents the occupancy status of a specific area [49]. These grids are fundamental to navigation frameworks like ROS2 Nav2 for navigation and obstacle avoidance. Consequently, many SLAM implementations produce occupancy grids to facilitate navigation. While traditionally two-dimensional, occupancy grids can be extended to three dimensions, referred to in this paper as *3D voxel maps*.

Since a robot's perception is inherently subject to measurement inaccuracies, integrating uncertainty into maps can be beneficial [50]. Many implementations of occupancy grids therefore store the occupancy probability of cells instead of simple binary states. These grids can be updated using various methods that account for measurement inaccuracies, including the Bayesian method [51].

Occupancy grids can be readily initialized with BIM data, but standard implementations rarely distinguish between permanent structures (derived from BIM) and temporary obstacles (detected by sensors). This distinction is potentially valuable for navigation. For instance, robots could treat objects matching BIM structures as static, always navigating around them. Conversely, obstacles that are not part of BIM can be more likely to disappear over time, so robots should consider routes blocked by these obstacles, as they might become accessible in the future. Standard occupancy grids merge these different obstacle types, losing valuable semantic information that can be inferred from the BIM model.

Existing BIM-enabled maps often treat BIM-derived elements as static. For instance, Hu et al. [52] present a comprehensive approach integrating robotics, LiDAR mapping, localization, and IoT sensing. Their system employs LeGO-LOAM for initial mapping followed by AMCL for localization with a static map that does not dynamically update to reflect environmental changes. While their system focuses primarily on visualizing sensor data (e.g., air quality monitoring) rather

than BIM-aware mapping, it illustrates a common pattern in construction robotics where maps remain largely fixed after initial creation. Like many existing approaches, it does not include semantic data or distinguish between obstacles that correspond to BIM elements and those that do not.

Gopee et al. [25] explored semantic maps derived from BIM data to enhance navigation. Their approach stores BIM object metadata directly within the map, enabling navigation systems to utilize object-specific information. However, their work neither addresses the distinction between BIM-verified obstacles (present in both BIM and sensor data) and sensor-only obstacles nor develops methods to verify correspondence between obstacles and BIM elements, focusing instead on representing and utilizing semantic information.

In subsequent work, Gopee et al. [53] extend their methodology with schedule-aware obstacle maps that consider construction progress at deployment time. They leverage *IfcWorkSchedule* Industry Foundation Classes (IFC) elements to account for ongoing construction activities. However, this approach relies on accurate schedules within the BIM model. Unexpected obstacles or scheduling changes are not typically featured in BIM, challenging systems that rely on predefined maps and schedules. This suggests the need for methods that determine construction progress based on real-time sensor observation to complement BIM information.

Addressing this limitation, Torres et al. [44] extend their localization work to detect changes between BIM and the real environment. They use signed distance computation to distinguish between BIM elements and new objects, apply DBSCAN to cluster detected new elements, and convert them into mesh representations via voxelization. Their method demonstrates how to differentiate between as-designed and other elements while preserving the semantics of the original model.

Similarly, Kavaliauskas et al. [9] recognize BIM objects in point clouds by counting points near each plane face using point-to-plane distance, and marking faces as present when sufficient points are present. Their method detects both complete and incomplete objects even in occluded and noisy conditions. However, point-to-plane distance may not properly account for LiDAR noise patterns, specifically distance noise that dominates at long distances. Moreover, relying on simple thresholds (the number of points near a face) to determine object presence may lack robustness, as it ignores the probabilistic nature of sensor data and possible variations in point cloud density.

Embracing a probabilistic perspective, Meyer et al. [41] present a point cloud-based progress monitoring approach. They address both change detection in point clouds and verification of scans against BIM. In contrast to Kavaliauskas et al., Meyer et al. leverage a probabilistic approach for managing uncertain evidence in the Dempster–Shafer theory (DST). This demonstrates how uncertainty modeling can improve differentiation between BIM structures and other obstacles. However, while DST offers theoretical advantages over Bayesian approaches commonly used in occupancy grids for handling ignorance and conflicting evidence, its higher computational complexity limits its use in real-time applications requiring efficient map updates.

2.3.3. BIM-guided navigation

Navigation systems are reliant on maps to both plan and execute paths. As discussed previously, BIM integration has the potential to enhance mapping through various methods. Integration approaches typically follow one of two paths: initializing standard navigation maps from BIM geometry or leveraging BIM's semantic structure directly.

The first approach, commonly used in robotic monitoring systems described in Section 2.2, converts BIM geometry into occupancy grids compatible with established navigation frameworks [27–29]. These systems typically employ either the ROS Navigation Stack or Nav2, based on ROS or ROS2 respectively. Zhai et al. [28] and Liao et al. [29] convert BIM to ROS occupancy maps but do not discuss navigation

stack configuration details. Park et al. [27] implement Boston Dynamics' GraphNav, which does not directly utilize BIM for navigation. Ibrahim et al. [42] use a SLAM-generated map aligned with BIM, with A* algorithm for planning. Consequently, their path planning is based on the SLAM-generated map rather than a map derived from BIM itself. These geometry-focused methods typically treat all obstacles identically. This limits their ability to adapt navigation behaviors based on obstacle characteristics.

Beyond these approaches, researchers have developed navigation methods that directly leverage BIM's semantic structure. For instance, Palacz et al. [54] developed a path planning methodology based on hypergraphs constructed from IFC data, encoding spaces as nodes and interconnections as edges. To account for bidirectional movement, they construct a directed hypergraph where doorways have asymmetric traversal costs that depend on movement direction. For path optimization, they propose a modified A* algorithm adapted for hypergraphs.

While high-level semantic planning efficiently navigates between spaces, it lacks granularity for obstacle avoidance within individual rooms. Addressing this limitation, hierarchical approaches integrate space-level planning with local planning. For instance, Hamieh et al. [55] present BiMov, a graph approach operating on three levels: a macro level for building topology, an external level for navigating around permanent structures, and an internal level for smaller obstacles including furniture and equipment. They employ Dijkstra's Algorithm for graph optimization and consider space accessibility by accounting for schedule-dependent changes such as maintenance. Similarly, Liu et al. [56] created a two-step system where BIM spaces form a network for high-level planning before switching to A* algorithm for granular path planning within individual spaces.

Navigation in dynamic environments such as construction sites is made difficult by changing layouts and unpredictable personnel movements. Effective BIM-guided navigation must therefore balance static BIM information with adaptability to changing conditions. The navigation approaches discussed, whether converting BIM to standard navigation maps or leveraging BIM's semantic structure, each offer different trade-offs in environments where both permanent and temporary elements coexist.

2.4. Inspection and analysis of collected data

Effective robotic data collection in built environments can be bolstered by two key capabilities beyond basic navigation: (1) next-best-view (NBV) planning, which optimizes sensor positioning for data capture, and (2) analysis methods that interpret the collected data.

NBV planning involves choosing viewpoints that maximize information gain based on task objectives. For inspection tasks, this could mean optimizing viewpoints for specific building components, while, for a mapping task, it might mean achieving broad coverage without blind spots. In built environments where obstacles and occlusions are prevalent, viewpoint selection is valuable for collecting visual data (with cameras) or spatial data (e.g., with LiDAR). NBV planning is a well-established research area [57], typically implemented through search or sampling-based methods [58]. These approaches generate candidate viewpoints, evaluating each for information gain (often using ray casting to compute visibility), and then selecting the best candidate as the next best view. Sampling-based methods are particularly useful where the environment is known fully or partly in advance. However, while predictable and well-understood, sampling-based methods have limitations [59]. They require numerous candidates to sufficiently explore the solution space, increasing computational costs, yet may still miss the best viewpoint if sampling is insufficient.

Once optimal viewpoints are determined and data is collected, the robot must analyze the data to assess object status. Analysis methods generally fall into two categories: occupancy-based and appearance-based [11]. Occupancy-based methods compare geometric data (point

clouds or depth images) directly with as-designed BIM models [60], excelling at detecting components constructed in a single stage. The primary requirement is precise registration between the collected as-built data and as-designed data. This can be challenging in dynamic environments where complete geometric coverage is difficult to obtain.

Appearance-based methods rely on the visual characteristics of objects to determine their status [61]. This makes them suitable for objects with multiple construction stages not modeled in BIM. Appearance-based methods mainly utilize deep-learning models that require labeled datasets and training for the set of relevant construction objects. Appearance-based methods are effective, but suffer from the lack of labeled datasets and pretrained models useful for construction inspection and monitoring [62].

Recent advances in vision-language models (VLMs) offer a promising alternative to appearance-based methods. Unlike conventional deep learning approaches that require extensive task-specific training data, VLMs leverage large-scale pretraining to enable zero or few-shot learning for novel tasks [63]. VLMs can also parse and reason about BIM metadata, where object descriptions are typically stored in natural language. This allows them to understand object characteristics and then identify these objects in images without specialized training. However, VLMs struggle with precise object localization within images [64], and may confuse analysis when multiple instances of the same object type appear in a single image.

2.5. Limitations and gaps

Automated monitoring approaches in built environments often involve a compromise between coverage and precision. Tag-based systems [33,65,66], for example, can track materials but generally cannot verify correct installation. Direct environmental sensing methods using photogrammetry or LiDAR [10,36,67] often require clear line-of-sight to targets. This presents challenges in cluttered construction environments [12,13] where obstructions are common. Moreover, the large data volumes generated by these sensors can complicate timely extraction of relevant information [40], although this issue can be alleviated with computer vision or point cloud processing techniques [6,11,41].

Robotic systems offer a promising avenue to address many of these data acquisition limitations by enabling automated, targeted data collection. Indeed, research has advanced individual components for BIM-driven robotics in built environments, including techniques for BIM-based localization [17,20,21], mapping [22], and navigation planning [23–26]. However, existing approaches often focus on isolated aspects and involve simplifying assumptions, rather than being presented as end-to-end robotic systems that handle the combined challenges of real-world situations.

Critically, many BIM-integrated robotic systems show limited adaptability to environmental changes. For instance, stop-and-go mapping approaches [27,28] often optimize scan positions using BIM but lack mechanisms to dynamically adjust these predetermined positions or inspection targets for unexpected obstacles or structural deviations from the model. Similarly, while some object inspection systems [16, 29] offer minor viewpoint adjustments, these are often limited to 2D representations and are insufficient for significant occlusions or discrepancies between the BIM model and actual conditions on-site.

A fundamental issue is the common assumption that BIM provides a static and accurate representation of the environment, causing difficulties in localization and mapping [46]. This is especially problematic on construction sites where structures may be partially complete or differ from design, affecting navigation. Multiple systems using BIM as part of a robotic navigation system [27–29,42,52] lack effective mechanisms to dynamically validate BIM elements against sensor data or update the robot's understanding when discrepancies arise. Moreover, the environmental maps used in these approaches do not distinguish between permanent BIM-defined structures and dynamic non-BIM obstacles.

This means these systems may be unable to account for obstacles defined in BIM when sensor data indicates their absence. A key concern is that these approaches are often evaluated in environments with strong BIM correspondence or few dynamic elements, so their robustness in more realistic dynamic conditions remains unclear.

Furthermore, BIM-driven navigation approaches focusing on high-level semantic planning based on BIM's topological structure [23,24, 54] typically do not account for robot-specific kinematic constraints or provide dynamic obstacle avoidance. Conversely, systems implementing robotic navigation with BIM-derived maps [29,52] rarely detail how path planning, interaction with dynamic obstacles, or BIM discrepancy handling are managed at the motion planning level. These approaches may overlook the need for efficient dynamic replanning in cluttered, changing environments or the potential benefits of differential obstacle costs that arise from a semantic interpretation. These limitations hinder the development of autonomous robotics for reliable task execution (e.g., detailed object inspection or facility condition assessment) via adaptive navigation and interaction in complex, changing built environments.

3. Object inspection with a quadruped robot

This section presents the methodology and implementation of our automated robotic inspection system for dynamic built environments. We begin with a system overview (Section 3.1) and complete inspection workflow (Section 3.2), followed by details of the hardware platform and software architecture (Section 3.3). The core technical contributions are then presented: the BIM-initialized SLAM and 3D voxel mapping systems (Section 3.4), the navigation approach (Section 3.5), and the integration of next-best-view planning with a vision-language model (Section 3.6). Finally, Section 3.7 outlines the experiment design to test the integrated system.

3.1. Overview

This paper presents an automated robotic inspection system that verifies construction object installation status by combining BIM, autonomous navigation, and computer vision techniques. The system merges the BIM's digital representation of building components with real-time sensor observations, enabling several capabilities:

1. **Robot Localization:** SLAM initialized with BIM geometry determines the robot's position and orientation relative to the BIM model.
2. **Adaptive Obstacle Mapping:** A 3D voxel map combines BIM data with real-time sensor observations, storing occupancy probabilities and semantic information (e.g., BIM-defined, sensor-confirmed, recently observed). This informs a navigation costmap, with different costs assigned to obstacles based on their classification.
3. **Viewpoint Selection:** Optimizes robot poses that attempt to maximize target object visibility based on the 3D voxel map, combining both BIM and sensor data.
4. **Navigation:** A hierarchical navigation stack uses kinematically feasible, cost-aware planning that balances travel distance with traversal costs. This enables the robot to avoid confirmed static obstacles while remaining flexible about temporary or unverified elements, allowing efficient navigation despite dynamic obstacles and BIM discrepancies.
5. **Object Verification:** Vision-language models analyze images captured at the chosen viewpoints, comparing target objects against their textual BIM metadata to verify installation status.

To achieve these capabilities, we integrate established robotics algorithms and software components with custom implementations. The architecture is depicted in Fig. 1.

3.2. System workflow

The system's workflow integrates several components to achieve autonomous inspection. The process, summarized in the steps below, begins with initialization and planning, proceeds through a main inspection loop for each target, and is supported by continuous mapping throughout. The following paragraphs describe this workflow in greater detail, with the underlying methodology for each component referenced in Sections 3.4–3.6.

1. Setup & Initial Planning

Input: BIM, Target List, Initial Pose

- Initialize **SLAM system** & coordinate frames
- Calculate initial optimal viewpoint for each target (**NBV Planner**, Section 3.6.2)

2. Main Inspection Loop

While targets remain:

1. Select the target based on shortest required travel distance (**Global Path Planner**, Section 3.5.1)
2. Navigate to current target's viewpoint
3. Inspect & refine viewpoint (up to 3 attempts):
 - Assume viewpoint & capture images
 - Evaluate viewpoint quality; if low ($> 20\%$ drop), recalculate & navigate to a new viewpoint
4. Analyze images (**VLM**, Section 3.6) & log results
5. Proceed to next target (start loop from the beginning) or if no targets are left, return to starting position

3. Continuous Operations

Running throughout the process:

- Update **3D Voxel Map** with LiDAR in real time
- Continuously evaluate feasibility of current path and if path is blocked, replan
- Classify obstacles detected with LiDAR based on their presence in BIM
- Adapt path planning costs based on classifications

The system starts by loading the **BIM model** of the building and a list of target objects. Additionally, the system requires the initial position and orientation of the robot relative to the **BIM model**. This initial pose is used for initializing the **SLAM system** and establishing the relationship between the robot's coordinate frame and the **BIM** coordinate frame. While this initialization pose need only be approximate, the **SLAM system** subsequently refines the robot's localization to achieve more precise relative positioning.

Initially, the **Next-Best-View (NBV) Planner** computes an optimal viewpoint for each target. Using a sampling-based approach (Section 3.6.2), the planner generates and simulates multiple feasible viewpoints, evaluating each with a cost function that maximizes visibility and attempts to fit the full object in the camera's field of view. The viewpoint with the highest score is selected for each target.

With viewpoints defined, the **Workflow Manager** determines the inspection order. It requests paths to close targets from the **Global Path Planner** (Section 3.5.1) and selects the target with the shortest path to minimize travel. Navigation to the selected viewpoint then commences. The **Global Path Planner** computes a path using information stored in

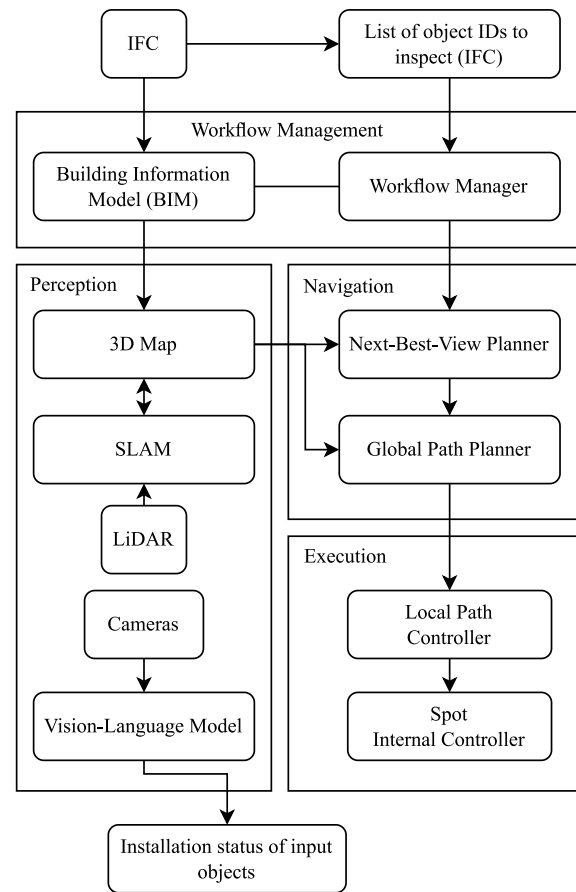


Fig. 1. System architecture diagram showing information flows between sub-systems.

the **3D voxel map**. This path is converted into waypoints by the **Local Controller**. If new obstacles block the route, the path is dynamically recalculated. This continues until the robot successfully reaches the target viewpoint or determines that the goal is unreachable.

Upon reaching a viewpoint, the robot adjusts its orientation, including pitch, to match the planned viewpoint. The **Workflow Manager** then triggers the cameras. The captured view is immediately re-evaluated; if its quality score has dropped by over 20% from the initial calculation (due to obstacles), the **NBV Planner** calculates a new viewpoint. This initiates another navigation and capture sequence. This refinement cycle repeats up to three times per target to efficiently acquire a usable image. All captured images are stored with metadata, including camera pose and target information.

After successful image acquisition for a target, the images are forwarded to a **Vision-Language Model (VLM)** for analysis (Section 3.6). The **VLM** compares the images against **BIM** metadata to determine if the object is installed or missing. Once the results are logged, the **Workflow Manager** selects the next closest uninspected target, and the entire cycle repeats until the inspection list is complete.

Throughout this process, the **3D voxel map** is continuously updated in real time from sensor data. This adaptive mapping is critical for operating in dynamic environments and directly informs the **NBV Planner**. Crucially, the map is semantic, classifying obstacles as: BIM-defined, BIM-corroborated by sensors, or new sensor-detected obstacles. These are assigned different costs for the **Global Path Planner**, enabling the robot to confidently avoid confirmed structures while navigating flexibly around unverified **BIM** elements (e.g., unbuilt walls) or temporary obstructions. This cost-aware approach ensures robust navigation despite significant discrepancies between the **BIM model** and the as-is site conditions.



Fig. 2. Boston Dynamics Spot robot used in this study, equipped with Ouster OS0 LiDAR for localization and mapping, and Intel RealSense D415 for capturing images of target objects.

3.3. Hardware and software

This section discusses the physical hardware and software that enable the capabilities outlined in Section 3.1, with subsequent sections addressing the methodology and implementation details.

3.3.1. Hardware

Our system utilizes the *Boston Dynamics Spot* quadruped robot platform, depicted in Fig. 2. This robot is equipped with a suite of built-in sensors, including an RGB camera, five monochrome cameras, six depth cameras, and an infrared camera. These integrated sensors support the robot's mobility functions and internal obstacle avoidance.

To support localization, mapping, and the capture of the inspection images, we equipped the robot with two additional sensors:

1. An *Ouster OS0* LiDAR, mounted using custom 3D-printed brackets, serving as the primary sensor for navigation, mapping, and localization (SLAM). It features 64 beams with a 90-degree vertical and 360-degree horizontal field of view and provides detailed point cloud data of the environment.
2. A *RealSense D415* RGB-D camera that provides images for inspection of target objects.

For computing, the system uses an onboard, passively cooled Linux computer for real-time localization and data processing, paired with an off-board Linux desktop (*Intel i5-14400 CPU*, 64 GB DDR5 RAM) for computationally intensive tasks, such as mapping, NBV planning, and path planning.

Communication between the onboard and off-board computers is maintained via a WiFi 6 connection involving the *Intel Wi-Fi 6E AX210* adapter onboard the robot, *Ubiquiti UniFi U7 Pro* Access Point, and *Mikrotik RouterBOARD hEX S* router.

3.3.2. Software

Our system is built on top of ROS2 Iron and Ubuntu 22.04. ROS2 is a modular and extensible framework that enables structured communication between different components of our system. The architecture follows a node-based approach, where individual software components share a common data pool through a topic-based publish-subscribe mechanism. This modularity and emphasis on code reuse



Fig. 3. BIM model used for both simulated (Unreal Engine 5) and real-world experiments.

in ROS2 allows us to leverage existing implementations of common robotics needs and integrate them with custom components and other libraries. Custom components specific to our research are implemented as ROS2 nodes with C++ for performance-critical parts and Python for higher-level tasks.

For navigation and control, Nav2 framework on ROS2 is used, combining *BT Navigator*, *Costmap 2d*, and *SMAC Planner*. Localization is achieved through *Kitware LiDAR SLAM*, while environmental mapping is handled by a custom 3D voxel mapping implementation with ROS2 integration. *IfcOpenShell* is used for BIM integration and analysis, and *Open3D* for point cloud processing and raytracing operations. Additionally, *PySwarms* and *Numpy* support optimization and data processing tasks, including NBV planning optimization and matrix calculations. Data processing and visualization were completed with *Matplotlib* and *Rviz2*.

For simulation, we leverage *Unreal Engine 5*, with several custom C++ components to support the simulation of perception and robot control, detailed in Section 3.7.1.

3.4. Localization and environmental representation

For mobile robots to operate effectively in construction settings, precise positioning relative to the site's Building Information Model (BIM) is essential. However, sensor drift in odometry and SLAM accumulation of inaccuracies over long trajectories challenge this alignment. As discussed in Sections 2.3.1 and 2.3.3, integrating BIM with SLAM is necessary for consistent relative localization while providing environmental mapping that reduces exploration needs.

Our system uses the BIM as the initial map. We utilize a single BIM model for both simulated and real-world experiments (Fig. 3). To leverage BIM geometric information effectively, we developed a pipeline that filters the model using IFC class labels to retain structural elements (walls, floors, columns) while discarding non-structural components (furniture, fixtures, doors, decorative elements). The STEP (Standard for the Exchange of Product model data) geometric data of the remaining BIM elements is then converted into a dense point cloud (Fig. 4) and voxel-downsampled to 10 cm resolution.

Beyond localization, our system requires environmental mapping for robot navigation and next-best-view (NBV) planning. Navigation needs traversable space and obstacle information, while NBV planning benefits from three-dimensional data for viewpoint optimization.

We separate the map used internally by SLAM from navigation and NBV mapping since SLAM implementations use tightly integrated representations optimized for real-time feature tracking. This separation allows SLAM system replacement without affecting other components, requiring only that SLAM supports geometric comparisons between sensor measurements and BIM.

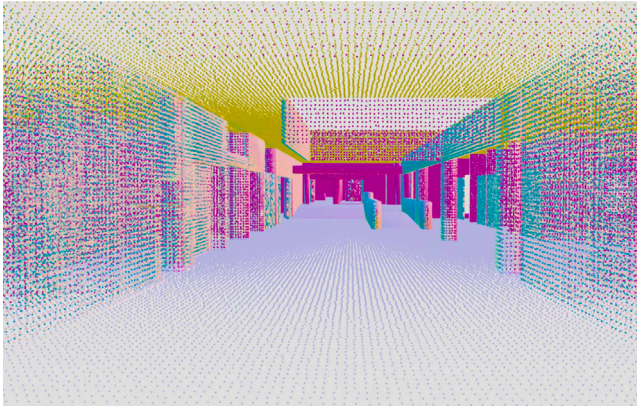
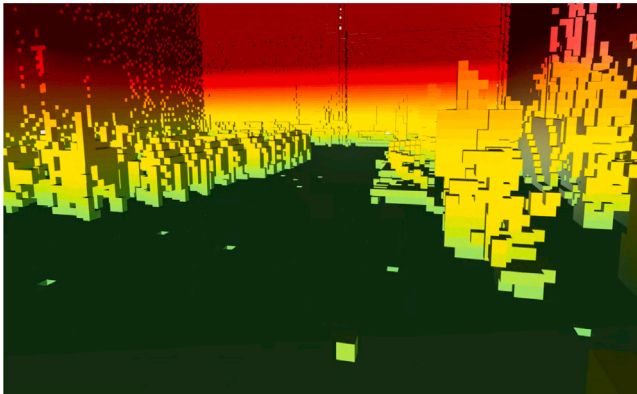


Fig. 4. BIM model, processed using IfcOpenShell to selectively extract its structural geometry to a point cloud using Open3D, which serves as initial data for navigation and SLAM.



(a) Spot robot during an experiment



(b) Visualization of the 3D voxel map of the same environment

Fig. 5. Example of the 3D voxel map showing (a) the Spot robot during an experiment and (b) the corresponding 3D voxel map (0.1 m resolution) of the same environment. This map serves as the primary environmental representation and ground truth for navigation and NBV planning. The map is initialized with BIM data and continuously updated with LiDAR observations to represent both known structure and other obstacles discovered by the robot.

With this in mind, our system utilizes three complementary maps: (1) internal SLAM representation; (2) primary 3D voxel map integrating BIM and observed obstacles; and (3) derived 2D costmap for navigation. The 3D voxel map enables NBV planning through volumetric modeling of free/occupied space. For adaptive navigation, the map

is required to support traversal costs to distinguish between different obstacle types. For this purpose, we flatten the 3D voxel map to a 2D costmap, which can directly be used in cost-aware path planning. A 2D representation for navigation is both sufficient and advantageous in our setting: it is computationally efficient, easily visualized, and compatible with standard robotics frameworks (Nav2) and path planners (SMAC Planner).

3.4.1. Localization

We selected 3D LiDAR SLAM for BIM-integrated localization based on compatibility with processed BIM geometry, robustness to construction challenges, and sensor compatibility. We evaluated multiple paradigms: Visual SLAM struggles with challenging lighting and poor visual features common on construction sites, and visual features are not directly compatible with BIM geometry. While 2D LiDAR SLAM is common in mobile robotics, its single scanning plane struggles to detect building structures like walls when floor-level obstacles cause occlusion, preventing accurate BIM-referencing.

Consequently, we selected 3D LiDAR SLAM as our localization approach. This choice offers several advantages in our context: (1) it captures detailed 3D point clouds that enable direct geometric comparison with structural BIM components; (2) it remains relatively unaffected by obstacles occluding walls by sensing above floor-level clutter; (3) it provides reliable performance even in visually challenging environments with poor lighting or poor features; and (4) it aligns well with the available sensors installed on our robot. Crucially, the geometric correspondence between 3D LiDAR data and 3D BIM elements facilitates consistent localization relative to the as-planned model.

After evaluating *Cartographer*, *hdl_graph_slam*, and *lidar_slam_ros2*, we selected *LiDAR SLAM by Kitware* for its ability to initialize directly with point cloud data from BIM-derived reference geometry. This implementation uses Iterative Closest Point (ICP) to compare captured LiDAR point clouds against its internal map representation supporting multiple feature types.

From the available feature types, we utilize edges and planes features, well-suited to both environment and BIM reference. Since BIM models consist primarily of planar surfaces (walls, floors, ceilings), we initialize the SLAM's plane map with a BIM-derived point cloud while leaving the edge map empty. BIM data acts as a prior soft constraint, guiding but not forcing building structure alignment. The system dynamically adds new features based on sensor observations, adapting to differences between as-designed BIM and actual site conditions.

This approach requires approximate initial robot location, a common limitation of LiDAR-based localization affecting deployment convenience. While global localization would eliminate manual setup, structural symmetry in indoor environments makes this challenging with LiDAR alone. Once initialized, our system performed robustly and achieved precise positioning despite significant BIM-reality differences.

3.4.2. 3D voxel map

As established in Section 3.4, our system maintains three environmental representations. This section details the primary representation, a probabilistic 3D voxel map that combines data from both LiDAR sensors and the BIM model. While inspired by OctoMap [68], we implemented a chunk-based solution to integrate and verify BIM data alongside sensor measurements. Our implementation enables direct embedding of BIM-specific information within voxels, supports specialized update logic for BIM structure verification (Algorithm 1), and stores per-voxel timestamps for the timed decay mechanism used in navigation costmaps (Section 3.4.3).

The map employs Bayesian updates to maintain occupancy probabilities, accounting for measurement inaccuracies while maintaining beliefs about the environment's state. BIM initialization minimizes exploration needs by converting IFC files into point clouds (Section 3.4). Rather than directly setting occupancy probabilities, voxels containing BIM points are marked with a special flag. An example map is visualized in Fig. 5, depicting the robot during an experiment and the corresponding map.

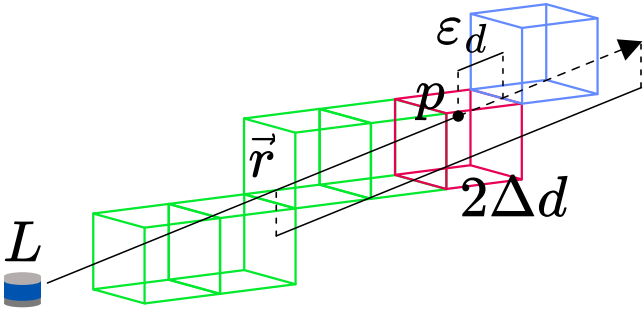


Fig. 6. Illustration of ray-based map update and BIM structure verification in 3D.

Base implementation. The voxel map employs chunk-based spatial partitioning with $16 \times 16 \times 16$ voxels per chunk at 0.1 m resolution. Chunks are stored in a hash map, indexed by their spatial position, enabling $O(1)$ access to any voxel in the environment. This allows for a sparse representation where memory is allocated only for observed regions. Chunks are initialized on demand when first accessed. Coordinate transformation between world and chunk spaces leverages bit-shifting operations since chunk dimensions (2^4 voxels) enable computationally efficient position mapping.

Internally, the voxels within each chunk are stored in contiguous arrays using z-major indexing, with occupancy probabilities initialized to $p_0 = 0.05$ (generally unoccupied belief).

For computational efficiency, we use log-odds values rather than raw probabilities. The relationship $L = \log \frac{p}{1-p}$ converts Bayesian updates into a simple additions. Specifically, if we denote the odds ratio as $\frac{p(x)}{1-p(x)}$, the Bayesian update can be expressed in odds form and then simplified:

$$L_{t+1} = \log \frac{p_{t+1}}{1-p_{t+1}} = \log \frac{p_t}{1-p_t} + \log \frac{p(z|x)}{p(z|\neg x)} \quad (1)$$

$$= L_t + L_{\text{sensor}}$$

where x represents the occupancy state and z represents the measurement.

Algorithm 1 describes the specialized voxel map update implementation. The method processes LiDAR point clouds by casting rays from the sensor origin and stepping through the 3D grid at intervals equal to the voxel resolution (0.1 m). Voxels along the ray path have their occupancy probability decreased, while voxels containing endpoint points have it increased. The algorithm also incorporates special handling for BIM structures, which we detail later on in this section.

The updates employ an asymmetric sensor model L_{sensor} . We define the prior probability of occupancy assigned to unobserved cells as $p_0 = 0.05$. For cells where obstacles are detected, we use an occupancy probability of $p_{\text{occ}} = 0.6$, while cells observed as free space are updated using an occupancy probability of $p_{\text{free}} = 0.47$. These parameters transform into log-odds updates following:

$$L_{t+1} = L_t - \log \frac{p_0}{1-p_0} + \begin{cases} \log \frac{p_{\text{occ}}}{1-p_{\text{occ}}}, & \text{if occupied} \\ \log \frac{p_{\text{free}}}{1-p_{\text{free}}}, & \text{if free} \end{cases} \quad (2)$$

In practice, these log-odds terms evaluate to constants: $L_{t+1} = L_t + L_{\text{occ}}$ or $L_{t+1} = L_t + L_{\text{free}}$. The asymmetric model prioritizes obstacle detection safety by making the system reluctant to clear previously detected obstacles.

We continuously update the voxel map with real-time LiDAR observations using the position estimates provided by SLAM (Section 3.4.1) to maintain proper alignment between sensor data and BIM coordinate frame.

Algorithm 1 BIM-Aware Voxel Map Update

```

1: Input:
2:   Voxel grid  $V$ 
3:   Point cloud  $P$ 
4:   Sensor origin  $O$ 
5: Constants:
6:   BIM search distance  $\Delta d$ 
7:   Resolution  $r$ 

8: for each endpoint  $p \in P$  do
9:    $\vec{d}_{ir} \leftarrow \frac{\vec{p}-\vec{O}}{|\vec{p}-\vec{O}|}$ 
10:   $d_p \leftarrow ||p - O||$ 
11:   $\text{BIM voxel} \leftarrow \text{none}$ 
12:   $d_{\text{BIM}} \leftarrow \infty$ 
13:   $\vec{c}_{ur} \leftarrow O$ 
14:  while  $||\vec{c}_{ur} - O|| \leq d_p + \Delta d$  do
15:     $v \leftarrow$  voxel at position  $\vec{c}_{ur}$ 
16:     $d_{\text{cur}} \leftarrow ||\vec{c}_{ur} - O||$ 
17:    if  $d_{\text{cur}} < d_p$  then
18:       $v.L \leftarrow v.L + L_{\text{free}}$ 
19:       $v.\text{timestamp} \leftarrow$  current time
20:    end if
21:    if  $|d_p - d_{\text{cur}}| \leq \Delta d$  then
22:      if  $v.\text{BIM flag}$  then
23:         $\vec{v}_{\text{center}} \leftarrow$  center coordinates of voxel  $v$ 
24:         $d_{vp} \leftarrow ||\vec{v}_{\text{center}} - p||$ 
25:        if  $d_{vp} < d_{\text{BIM}}$  then
26:           $\text{BIM voxel} \leftarrow v$ 
27:           $d_{\text{BIM}} \leftarrow d_{vp}$ 
28:        end if
29:      end if
30:    end if
31:     $\vec{c}_{ur} \leftarrow \vec{c}_{ur} + \vec{d}_{ir} \cdot r$ 
32:  end while
33:  if  $\text{BIM voxel}$  then
34:     $\text{BIM voxel}.L \leftarrow \infty$ 
35:     $\text{BIM voxel}.timestamp \leftarrow$  current time
36:  else
37:     $v_p \leftarrow$  voxel containing endpoint  $p$ 
38:     $v_p.L \leftarrow v_p.L + L_{\text{occ}}$ 
39:     $v_p.timestamp \leftarrow$  current time
40:  end if
41: end for

```

BIM structure verification. To assign different navigation costs to different obstacle types, we distinguish between BIM-defined structures and other obstacles through the map update process (Algorithm 1 and Fig. 6).

During initialization, voxels that should contain BIM structures are marked with a special flag, while maintaining the default occupancy probability ($p_0 = 0.05$). As illustrated in Fig. 6, the LiDAR sensor (L) emits a ray ($\vec{r}$) intersecting a surface at hit point (p), with the corresponding voxel marked in red. To compensate for distance errors (ϵ_d) from both LiDAR inaccuracy and SLAM localization, the system examines a $\Delta d = \pm 0.2$ meter segment of the ray extending bidirectionally from p along the ray $\vec{r}$. Through Bayesian updates, the system increases the occupancy probability of the red voxel containing the hit point while decreasing the probability of green voxels along the ray's path (assumed to be free space). The blue voxel represents a BIM structure, and if such a voxel lies within the $2\Delta d$ segment, the occupancy probability of the closest blue voxel is also increased.

When a LiDAR hit coincides with a BIM-flagged voxel (blue in Fig. 6) within the search distance ($2\Delta d$ segment), we set its occupancy probability to 100%, confirming that portion of the structure exists. This check enables the system to verify which modeled structures have been constructed or exist in the environment.

Notably, verified BIM structures remain subject to standard occupancy update rules: subsequent rays passing through without hitting obstacles decrease occupancy probabilities, allowing the map to adapt even if structures on the site are later removed.

For each voxel update, the system updates a timestamp recording when each voxel was last interacted with. These three attributes (occupancy probability, BIM flag, and observation timestamp) are processed to generate the navigation costmap (Section 3.4.3), which enables informed decisions about obstacle traversability, distinguishing between verified structures, temporary obstacles, and free space.

3.4.3. Navigation costmap

We project our 3D voxel map into a 2D costmap following Nav2 framework conventions.

Implementation. For each costmap cell (x, y) , we examine voxels within a height range relative to the robot's position. The lower bound is set one voxel above the ground plane estimate (estimated by the robot internally) to avoid noise from ground readings. The upper bound is set three voxels above the robot's body height, accounting for the LiDAR sensor extending above the robot, to ensure avoidance of overhead obstacles that could collide with the LiDAR or the robot.

Within this vertical column of voxels, we track four attributes: the maximum occupancy probability encountered, whether any voxel has the BIM flag set, whether the voxels have been modified during a LiDAR update, and a timestamp representing the latest such update.

The system establishes a crucial distinction between verified BIM structures and temporary obstacles through its cost assignment. This assignment logic includes our timed cost decay approach for temporary obstacles. When multiple conditions apply, the one with highest cost is selected:

- **Lethal cost (100):** Applied when a BIM-flagged structure has been observed (occupancy probability > 50% with BIM flag) or when a temporary obstacle has been recently observed (occupancy probability > 50% and seen within the last 10 s).
- **Medium cost (50):** Applied to obstacles with occupancy probability > 50% that have not been seen within the last 10 s. This condition implements the timed cost decay, where the cost of temporary obstacles is reduced after 10 s of not being observed.
- **Low cost (25):** Applied to BIM-flagged structures that are considered unverified until modified by sensor data (BIM flag present but no sensor updates).
- **Free space (0):** Applied when occupancy probability $\leq 50\%$.

Our approach to dynamic environment handling is implemented by this cost assignment. Verified BIM structures remain lethal indefinitely, as these are thought to represent permanent building elements that the robot should never attempt to traverse. In contrast, temporary obstacles (like equipment or materials) are only considered lethal for 10 s after their last observation. After this period, the timed cost decay reduces their cost to medium, allowing the path planner to consider routes through these areas if no better alternatives exist. Moreover, with sufficient sensor evidence confirming the space is free, the cost of obstacles will eventually drop to zero through the 3D voxel map updates, effectively removing them from the costmap. This happens regardless of their original source (BIM or sensor data). This approach balances safety with flexibility: the robot primarily seeks paths avoiding all obstacles but can “risk” going through areas where temporary obstacles were previously detected if necessary. These cost values were determined through testing and were found to provide satisfactory performance in our test environments.

3.5. Navigation

Navigation in dynamic built and construction environments requires systems that efficiently traverse between locations while adapting to both static structural elements and dynamic obstacles. Our

approach comprises: (1) Global Path Planning generating cost-aware movement trajectories that adapt to BIM-reality discrepancies, and (2) Local Control translating global paths into sequential waypoints executed through the robot's API.

The navigation system is implemented within the *Navigation2* (Nav2) framework, which includes the *Behavior Tree Navigator* for creating and monitoring navigation tasks, *Planner Server* for managing path planners, and *Costmap 2D* for maintaining costmaps. This enables us to leverage established solutions while adapting them to address the specific challenges relevant to our study.

3.5.1. Global path planning

Global path planning generates collision-free trajectories between inspection points or goal positions. Our approach utilizes the *SMAC planner* [69], a cost-aware path planning framework that optimizes for a balance between path distance and cost-based obstacle avoidance.

SMAC implements Hybrid-A*, improving over traditional A* by generating kinematically feasible plans accounting for obstacle costs while maintaining computational efficiency. This suits construction sites or other built environments where navigation must handle permanent BIM elements, BIM-reality discrepancies, and temporary obstacles.

Our implementation distinguishes obstacle types based on source and status from BIM and LiDAR observations. The planner receives costmaps where: verified BIM structures (confirmed through sensor observations, Section 3.4.2) receive lethal costs prohibiting traversal; unverified BIM elements receive non-lethal costs (avoided but traversable when necessary); recently observed non-BIM obstacles receive lethal costs that decay to medium cost in time.

This enables adaptation to BIM-reality discrepancies. For partially completed floors where BIM walls remain unbuilt, the robot can plan through unverified BIM elements rather than being artificially constrained. For temporary obstacles blocking routes, the system initially treats them as impassable but gradually allows planning through previously observed areas.

Implementation. We use Nav2's Planner Server with SMAC Planner from *nav2-smac-planner*. The planner utilizes a 2D costmap (0.05 m resolution) with two layers: Static Layer subscribing to our navigation costmap (Section 3.4.3), and Inflation Layer adding 0.55 m safety margins through exponential cost decay.

SMAC configuration uses the Reeds-Shepp motion model with a minimum turning radius parameter of 0.20 m for planning stability, even though the Spot robot itself is capable of rotating in place. To balance safety with computational efficiency, we set a maximum planning time of 10 s and a cost penalty of 100 to strongly discourage traversal through high-cost areas while maintaining the ability to pass through them when necessary. The cost penalty value of 100 was determined through testing in simulation, where we found that lower values (e.g., 50) provided insufficient avoidance of high-cost areas, while significantly higher values led to overly conservative behavior. This parameter may require environment-specific tuning in different environments, though our testing suggests 100 provides a reasonable balance between obstacle avoidance and path efficiency. For collision checking, the robot's footprint was modeled as a rectangular shape with dimensions of 1.3 m $\times$ 0.5 m, with slight front-back asymmetry to account for the robot's protruding arm (0.7 m forward, 0.6 m backward from the center point).

The navigation system plans a new path only when a new goal is defined or when the current path becomes invalid (has the robot encroaching on lethal obstacles). Progress checking ensures the robot moves at least 0.5 m or rotates at least 0.2 radians within a 10-second window, and triggers recovery behaviors when progress stalls. For navigation failures, the system implements recovery behaviors with up to two retry attempts, alternating between backing up 0.5 m and waiting 5 s to allow dynamic obstacles to clear before retrying path planning. Goal achievement requires meeting tolerances of 0.1 m for

position and 0.1 radians for orientation. Path planning requests are managed by Nav2's Behavior-Tree Navigator, which handles these recovery behaviors when initial paths become blocked or invalid.

In our implementation, deployed on hardware described in Section 3.3.1, the planner typically generates new paths in 50–200 ms in our environments. This level of responsiveness is sufficient for replanning when obstacles are encountered and for facilitating target sequencing in a reasonable time (Section 3.6.1).

3.5.2. Local control

Local control translates the planned paths into robot movement commands. Rather than implementing custom low-level control algorithms, our system utilizes the Boston Dynamics Spot robot's internal real-time obstacle avoidance and terrain adaptation. This division of responsibilities allows our system to focus on higher-level navigation and obstacle avoidance, which is practical, since direct leg control for the robot is not exposed by the manufacturer in the standard interface.

Implementation. The local control system interfaces between Nav2's global planning and the Spot robot's API via the Boston Dynamics SDK. The implementation follows these steps:

1. Receive the planned global path as a sequence of pose waypoints
2. Transform each waypoint into Spot's odometry frame for consistent reference
3. Track the robot's current position in the odometry frame at 5 Hz intervals
4. Send waypoints sequentially to the robot via the Boston Dynamics SDK
5. Advance to the next waypoint when the robot comes within 0.5 meters of the current target
6. If unable to reach a waypoint, escalate error to trigger recovery behaviors

Communication with the Spot robot is handled through the `synchro_se2_trajectory_point_command`, which instructs the robot to navigate to a specific (x, y) position with a desired heading. Our system maintains a waypoint queue and issues commands based on the robot's current position rather than waiting for an explicit completion signal. This approach provides a straightforward bridge between our navigation planning and the robot's locomotion capabilities.

3.6. Object inspection

Our system autonomously captures images to assess installation status of building components defined by target BIM IDs. This involves viewpoint planning, navigation, image acquisition, and analysis as outlined in Section 3.2. This section details workflow management, viewpoint planning, and image analysis methodologies.

3.6.1. Workflow management

The Workflow Manager coordinates the autonomous inspection process. Building upon the initial viewpoint generation by the NBV Planner (Section 3.6.2), it is responsible for determining the order of inspection, coordinating navigation to viewpoints, managing image capture, and initiating replanning if image quality is insufficient.

Target sequencing. To efficiently inspect multiple targets, the Workflow Manager employs a greedy nearest-neighbor algorithm ($O(n^2)$) as a computationally practical alternative to optimal Traveling Salesman Problem (TSP) solutions (e.g., $O(n!)$ for brute-force search).

The algorithm operates in two phases: first, from the robot's current position, it identifies the three closest unvisited target viewpoints based on Euclidean distance. Then, for these candidates, it requests paths from the Global Path Planner (Section 3.5.1) and selects the target reachable via the shortest valid path. This straightforward approach balances computational speed with sequence efficiency, as path planning is reserved for a small subset of targets.

Retries and failures. The system incorporates mechanisms for robustness. If the navigation system reports a target viewpoint as unreachable, the Workflow Manager removes it from the current inspection queue to prevent unproductive repeated attempts.

Furthermore, as initial viewpoints are based on the BIM, discrepancies with the as-built environment can affect view quality. To counter this, upon reaching a target viewpoint, the Workflow Manager evaluates the quality of the current robot pose using the same NBV cost function used in the generation of the initial viewpoint (described in Section 3.6.2). The system permits up to 20% quality degradation from the initially calculated score. When the score falls below this threshold, the Workflow Manager requests a new viewpoint for the current target. This replanning and navigation cycle can be attempted up to two additional times (for a total of three attempts per target object) to prevent loops.

Implementation. The Workflow Manager is implemented as a Python ROS2 node that communicates with the Global Path Planner for routing, the NBV Planner for viewpoint generation, and Nav2 for navigation execution. At each inspection position, it associates captured images with BIM metadata and current robot pose for storage. Upon mission completion, the robot returns to its starting position.

3.6.2. Next-best-view planning

Our use case employs a sampling-based NBV approach to determine a viable camera pose for observing known target objects. The planner operates on a 3D world model derived from the 3D voxel map (detailed in Section 3.4.2) that integrates both a priori BIM data and LiDAR observations. The aim of the algorithm is to find a viewpoint that maximizes target visibility while accounting for occlusions from building structures and other obstacles, which we formulate as a constrained optimization problem.

Problem formulation. The objective is to find a robot pose, P^* , that maximizes a view quality score, $S(P)$. A pose P is a rigid-body transformation representing the robot's position and orientation. The optimization problem is defined as:

$$P^* = \operatorname{argmax}_{P \in \mathcal{V}} S(P) \quad (3)$$

The quality score $S(P)$ is a composite objective function:

$$S(P) = \underbrace{\frac{N_{\text{target}}(P)}{N_{\text{total}}}}_{\text{Coverage Ratio}} + \underbrace{F(P)}_{\text{Frame-Fit Bonus}} \quad (4)$$

where $N_{\text{target}}(P)$ is the number of pixels occupied by the target object when viewed from the camera pose associated with P , determined via raycasting; N_{total} is the total number of pixels in the simulated camera frame (90×68); and $F(P)$ is a binary frame-fit bonus. Its value is 1 if a pre-sampled point set on the target's surface, generated via Poisson-disk sampling (256 points), projects entirely within the camera's image bounds, and 0 otherwise.

The rationale for this composite score is to manage the trade-off between two competing goals. The Coverage Ratio incentivizes closer views that maximize the pixels-on-target, while the Frame-Fit Bonus penalizes poses that are too close, ensuring the entire object is captured in the frame.

The set of valid poses, $\mathcal{V}$, is limited by the following constraints:

1. **Navigability and Reachability:** Pose feasibility is determined by two checks, both using the navigation costmap's (Section 3.4.3) lethal cost threshold (≥ 100). First, a reachability map is pre-computed each planning cycle using a flood-fill algorithm from the robot's current location, treating lethal cells as impassable. This enables efficient lookup to discard candidate positions in isolated or inaccessible areas (e.g., outside the building). Second, the robot's 2D footprint at pose P must not intersect any cell with a lethal cost.

2. **Pitch Limits:** The robot's pitch angle is clamped within the set limits of $[-10^\circ, 30^\circ]$.
3. **Search Area:** The robot's position must be within a 6-meter search radius of the target's center.

Optimization algorithm and implementation. We solve this optimization problem using Particle Swarm Optimization (PSO). The procedure is summarized in Algorithm 2. We generate 100 candidate positions distributed in a circular pattern around the target object within a 6-meter radius. This sampling uses a uniform angular distribution and square-root radius distribution to ensure uniform area coverage, providing initial positions for the PSO particles. The PSO algorithm then searches the 2D space (x, y position of the robot base) to optimize the viewpoint quality score $S(P)$. Each particle represents a candidate robot pose and its fitness corresponds to the calculated quality score. We use PSO parameters tuned for the problem (cognitive coefficient $c_1 = 0.5$, social coefficient $c_2 = 0.3$, inertia weight $w = 0.9$) and found that five generations were typically sufficient for convergence.

For each (x, y) position evaluated by PSO, the robot's yaw and pitch are adjusted to orient the robot so that the camera, offset from the robot base by a fixed transform $T_{R \leftarrow C}$, points at the target center, with pitch clamped to the allowed range. The robot's z-coordinate (height) is based on the robot's ground plane estimate.

The 3D scene for raycasting is constructed from two sources. The target is represented by its BIM mesh. The environment is represented by a mesh generated from the 3D voxel map: a corresponding mesh cube is created for each voxel that either (1) has an occupancy probability greater than 50%, or (2) has a BIM flag (occupied in BIM) but has not yet received a sensor update. Camera perspective uses a pinhole model with calibrated intrinsics and raytracing through a low-resolution (90×68 pixels) view frustum (maintaining the 640×480 aspect ratio used with the real camera). This resolution balances computational efficiency with simulation accuracy for viewpoint scoring. To address model, localization, and sensor inaccuracies, a 0.3-meter margin is applied around the target object. Voxels within this margin are considered part of the target and excluded from collision checks, preventing misclassification of nearby structures as obstacles. This trade-off increases robustness against common inaccuracies, but it may ignore actual obstacles very close to the target.

3.6.3. VLM-based verification and reporting

To determine whether an object has been properly installed, it is necessary to compare its captured image with its BIM requirements. One general source of information is the IFC class of each element. Some of the more self-explanatory classes include *IfcWall*, *IfcWindow*, and *IfcDoor*. However, there are elements that do not neatly fit into these categories and use a generic class label like *IfcBuildingElementProxy*. In such cases, we need to refer to the other properties, such as object name and description. This is a challenge for traditional computer vision approaches.

Conventional methods like object detection or semantic segmentation depend on extensive labeled training datasets covering all potential BIM object types and their variations. Gathering such a dataset would be prohibitive. Furthermore, such models would need to be combined with natural language processing (NLP) techniques or language models to determine a proper class from textual attributes.

We overcome these challenges by utilizing a vision-language model (VLM), which offers key advantages for this task. First, it can perform zero-shot classification without the need to gather specific training data. Second, it can directly interpret textual BIM metadata (including names, descriptions, and class information). Finally, it can reason about the visual evidence in captured images and whether the evidence satisfies the corresponding natural language descriptions.

We use the commercially available cloud-based model Claude 3.7 Sonnet by Anthropic, which interprets the object's nature from its metadata, and then determines its installation status from the provided image. We access the VLM through API calls from the off-board computer that communicates with the robot.

Algorithm 2 PSO-based Next-Best-View Planning

```

1: Input:
2:   Target mesh  $M_{target}$ , target center  $p_{target}$ 
3:   Environment mesh  $M_{env}$ , costmap  $C_{nav}$ 
4:   Robot's current pose  $P_{current}$ 
5: Constants:
6:   Number of particles  $k = 100$ , generations  $G_{max} = 5$ 
7:   Camera-to-robot transform  $T_{R \leftarrow C}$ 
8:   Camera intrinsic matrix  $K$ , resolution ( $90 \times 68$ )
9:   Pre-sampled target points  $P_{samples}$  (256 points)
10:  Pitch limits  $[\theta_{min}, \theta_{max}]$ 

11: function FINDNEXTBESTVIEW(Input, Constants)
12:    $R_{set} \leftarrow \text{FloodFill}(P_{current}.position, C_{nav})$ 
13:    $\mathcal{P} \leftarrow \text{InitializeParticles}(k, \text{around } p_{target})$ 
14:    $T_{g_{best}} \leftarrow \text{null}$ 
15:    $S_{g_{best}} \leftarrow -\infty$ 
16:   for generation  $g = 1$  to  $G_{max}$  do
17:     for each particle  $p_i \in \mathcal{P}$  do
18:        $params \leftarrow [p_i.position, p_{target}, T_{R \leftarrow C}]$ 
19:        $T_{robot} \leftarrow \text{PointCameraToTarget}(params)$ 
20:        $Pitch(T_{robot}) \leftarrow \text{Clamp}(Pitch(T_{robot}), \theta_{min}, \theta_{max})$ 
21:        $free \leftarrow \neg \text{FootprintCollision}(T_{robot}, C_{nav})$ 
22:        $reachable \leftarrow p_i.position \in R_{set}$ 
23:       if  $\neg(free \wedge reachable)$  then
24:          $p_i.score \leftarrow 0$ 
25:         continue
26:       end if
27:        $T_{camera} \leftarrow T_{robot} \cdot T_{R \leftarrow C}$ 
28:        $rays \leftarrow \text{CreateRayFrustum}(T_{camera}, K)$ 
29:        $hit_{target} \leftarrow \text{CastRays}(rays, M_{target}, M_{env})$ 
30:        $S_{coverage} \leftarrow \text{Count}(hit_{target}) / \text{Count}(rays)$ 
31:        $P_{proj} \leftarrow \text{ProjectPoints}(P_{samples}, T_{camera}, K)$ 
32:        $F_{fit} \leftarrow 1$  if  $\text{AllPointsInFrame}(P_{proj})$ , else  $0$ 
33:        $p_i.score \leftarrow S_{coverage} + F_{fit}$ 
34:       if  $p_i.score > p_i.best\_score$  then
35:          $p_i.best\_score \leftarrow p_i.score$ 
36:          $p_i.best\_pos \leftarrow p_i.position$ 
37:       end if
38:       if  $p_i.score > S_{g_{best}}$  then
39:          $S_{g_{best}} \leftarrow p_i.score$ 
40:          $T_{g_{best}} \leftarrow T_{robot}$ 
41:       end if
42:     end for
43:      $\text{PSOUpdateParticles}(\mathcal{P}, \text{Position}(T_{g_{best}}))$ 
44:   end for
45:   return  $T_{g_{best}}$ 
46: end function

```

Implementation. The process of determining installation status using the VLM consists of multiple steps. Each step builds upon the previous one and is sent as part of a single conversation. To maximize response consistency, we interact with the VLM API by overriding sampling temperature to zero, while leaving other settings at their defaults. However, it should be noted that even with this setting, the model does not provide perfectly deterministic results.

In the first step, we prompt the model to analyze only the BIM metadata:

```

You will be given metadata from a BIM (IFC)
↪ element. Your task is to analyze the
↪ metadata, and try to figure out what sort
↪ of element it might refer to.
Here's the metadata information of the first
↪ element:

```

{metadata}

Based on the information provided, what can you

- ↪ infer about the element? What type of
- ↪ element do you think it is? What are the
- ↪ key characteristics of the element? What
- ↪ are the key features that you would expect
- ↪ to see in an image containing this
- ↪ element?

This initial prompt encourages the VLM to interpret the metadata and identify key visual characteristics, independent of the image. It helps the model establish expectations about appearance, size, and typical placement based solely on BIM information. Importantly, we added this step because metadata can be in languages that are not well supported by the VLM (in our case, Finnish). By effectively translating the metadata into English at this stage, we aim to improve the performance of the subsequent classification.

The BIM metadata provided to the VLM for each target object is a structured text representation extracted from the IFC file. This includes the element's unique ID, its IFC class (e.g., *IfcFurnishingElement*, *IfcDoor*), a human-readable *Name*, *ObjectType*, and *Tag*. Furthermore, it includes *Property Sets* associated with the element. These property sets can contain information such as functional categories (e.g., "Furniture"), dimensional data (like *Area* or *Volume*, identity data (like *workset* information), phasing information (e.g., "New construction"), and *Family* and *Type* names that can provide crucial clues about the object's nature.

In the second step, we present the captured image along with specific guidance about how to approach the analysis:

Next, your task is to determine whether the

- ↪ given image contains the element and
- ↪ whether it can be assumed that the element
- ↪ is installed despite not being visible in
- ↪ the image.

Be aware that:

1. The object may be partially visible or
 - ↪ occluded by other items
2. It could appear at any scale, angle, or
 - ↪ position
3. The lighting/quality may vary
4. The object might be installed/mounted/
 - ↪ integrated into the scene
5. The image is pointed directly at the location
 - ↪ the element should be at. If you see a
 - ↪ similar element in the image, it is likely
 - ↪ that the element is present.

Please provide (in order):

- General observations about the image
- Step-by-step analysis of the image
- Key visual evidence that can be used to
 - ↪ support your decision
- Any assumptions you made

Finally, compare the image to the metadata

- ↪ extracted from the IFC, and give your
- ↪ assessment regarding the following two
- ↪ questions:

1. Is the element described in the metadata
 - ↪ ACTUALLY present in the image (we can
 - ↪ directly see the object in the image)?
2. Can we assume that the object described in
 - ↪ the metadata is installed in area
 - ↪ indicated by the image (even if we can not
 - ↪ directly see it)?

This second step asks the model to consider two key aspects. First, whether the object described in the metadata is visible in the image. Second, whether there is evidence that the object has been installed, even if not directly visible. This distinction is essential because some objects may be occluded by other elements. For example, a window might be hidden behind blinds, but the presence of the blinds itself suggests that a window is likely installed behind them.

Finally, in the third step, we request binary yes/no answers sequentially to enable programmatic processing of results:

Provide your final assessment on whether the

- ↪ element is ACTUALLY PRESENT in the image.
- ↪ Give a one word answer (yes/no).

Provide your final assessment on whether the

- ↪ element is INSTALLED in the area indicated
- ↪ by the image. Give a one word answer (yes
- ↪ /no).

This represents a practical approach to binary classification using generative models that do not provide probabilistic confidence values for the generated output. Should the VLM fail to provide a response that can be parsed (meaning any response other than "yes" or "no"), we assume the answer to be no.

3.7. Experimental design

This section describes our experimental methodology for evaluating the proposed system. We conducted two simulated experiments (20 trials each) testing system performance and a real-world validation experiment testing real-world applicability.

The evaluation focused on three key capabilities: robustness to dynamic obstacles, adaptation to discrepancies between BIM and reality, and ability to capture images of target objects to analyze their presence. These aspects test the navigation approach that treats objects differently based on their type (Section 3.5.1). Additionally, these aspects indirectly test our localization and mapping methods, including the verification of both BIM-defined structures and other obstacles (Sections 3.4.1, 3.4.2, and 3.4.3). Furthermore, they test the NBV planning and inspection functionality relevant to our use case. The quantitative metrics used are defined in Section 3.7.3.

To specifically quantify the impact of our core contributions, we also performed a comparative analysis against two ablated system variants. The first, a *Non-Semantic* variant, removed our custom BIM-aware mapping and cost logic. The second, a *No NBV Retries* variant, disabled the adaptive viewpoint refinement mechanism. These comparisons are detailed further in Section 4.2.

All simulated experiments leveraged a digital twin of the real-world test environment based on the BIM model. This digital twin facilitated controlled experiments by enabling the randomization of starting positions, obstacle locations, and the spawning and movement of virtual personnel.

In our real-world experiment, we conducted a single trial that followed the same protocol as the first simulation experiment. While multiple real-world trials would be ideal, the practical challenges of repeatedly conducting experiments in such environments limited our ability to perform multiple runs. Instead, we focused on validating that the system functions in the real world in an environment consistent with our simulation.

In all experiments, we used six distinct objects as targets to keep the experiments directly comparable with each other:

- (a) Electrical Door
- (b) Reception Desk
- (c) Fire Extinguisher 1
- (d) Fire Extinguisher 2
- (e) Fire Extinguisher 3
- (f) Large Window

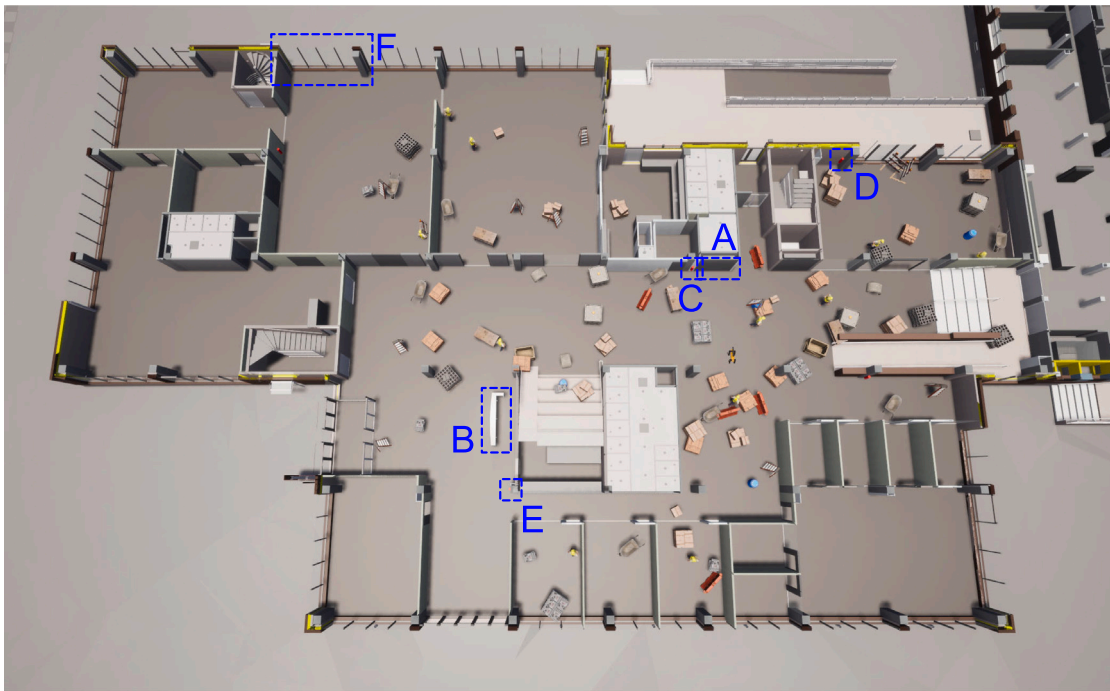


Fig. 7. Simulated Experiment 1: Baseline environment with randomly placed obstacles and moving construction workers. Target objects are marked and remain consistent across all experiments.

While the chosen target objects are typically not especially relevant from a construction progress monitoring perspective, we consider them representative of similar monitoring challenges: requiring viewpoint planning to capture targeted data and to ensure clear visibility. The objects were selected primarily because they were well-defined in the BIM model available to us and physically present in the real test environment. The inclusion of smaller objects like fire extinguishers deliberately challenges the system's precision in localization and image capture, which is similarly important for the inspection of other construction components.

The upcoming sections detail the setup of the simulation environment and simulated experiments, as well as the real-world experiment. We also describe how we collected and analyzed data to assess the system's performance.

3.7.1. Simulation experiments

To create a challenging and faithful simulation for testing the system, we developed a simulation based on Unreal Engine 5. Several factors drove this decision: Unreal Engine 5 provides real-time simulation, supports importing BIM models in IFC format via the Data-smith plugin, and includes features for simulating dynamic behaviors (e.g., moving obstacles and personnel).

To equip Unreal Engine for our simulation needs, we developed custom components for:

- **LiDAR simulation:** implementing raycasting from the robot's sensor position with noise characteristics estimated to match our physical sensor (Ouster OS0). The simulation includes multiple noise sources: Gaussian ray angular noise ($\sigma = 0.1^\circ$) in both horizontal and vertical directions to simulate beam spread; hit point angular noise ($\sigma = 0.5^\circ$) around the sensor's up-axis to perturb detected point positions; and distance noise ($\sigma = 0.1$ cm) along ray direction to simulate distance measurement inaccuracies. Additionally, we modeled range-dependent dropout with a linear increase from 0% at close range to a 90% point loss at 35 m, with a maximum detection range of 50 m.

- **Camera simulation:** generating RGB images using Unreal Engine 5's rendering capabilities, with camera parameters matching our physical camera's intrinsics. The virtual camera was positioned with the correct offset from the robot's body.
- **Robot control:** receiving motion commands from ROS2 and controlling the robot body.

To ensure consistency with the real-world experiment, the simulation environment was created using the BIM model of our real-world test building. Additionally, we ran the same high-level navigation and mapping software stack in both the simulated and real-world experiments to ensure transferability. However, a key limitation of our simulation is the absence of Boston Dynamics Spot's proprietary low-level locomotion controller, which provides an additional layer of real-time obstacle avoidance and terrain negotiation. In simulation, we implemented steering behaviors that followed the path computed by the global planner while maintaining velocity constraints similar to the physical robot. We consider this simplification appropriate for evaluating the high-level behavior and cost-aware decision-making of our navigation system. The global planner alone enables kinematically feasible obstacle avoidance, although additional lower-level planning (i.e., Spot's internal controller) would improve overall robustness.

We conducted two simulated experiments, each designed to test specific aspects of our system:

Simulated experiment 1: Dynamic environment navigation. The first experiment tested the system's ability to navigate in dynamic construction environments with moving personnel and changing obstacles. For each of the 20 trials, the environment contained 80 randomly positioned construction-related obstacles such as cardboard boxes, bricks, equipment, and ladders. Additionally, 10 virtual construction workers continuously navigated between random positions in the environment, simulating unpredictable human traffic found on actual construction sites. To add further challenges, one randomly selected obstacle was relocated every second to a new position at least 2 m away from the robot. The simulation environment and the positions of target objects are depicted in Fig. 7.



Fig. 8. Simulated Experiment 2: Environment with BIM discrepancies. Red boxes indicate added static obstacles and blue boxes highlight removed BIM structures, both relative to Experiment 1.

Simulated experiment 2: BIM-reality discrepancy. The second experiment examined the system's ability to adapt to discrepancies between the BIM model and the actual environment. We modified the environment by adding walls not present in the BIM model, blocking potential routes. Simultaneously, we removed several internal walls (to simulate a situation where they were "not yet built"), challenging the robot to take shortcuts through these openings. This environment and the changes are shown in Fig. 8. Critically, the modifications were designed so that the only viable path between certain targets required traversing through a location where the BIM indicated a wall, but none existed in the simulation. This demonstrates how the system detects when BIM elements are not supported by sensor data and how it adapts path planning according to the in-progress conditions.

3.7.2. Practical experiment

The real-world experiment was conducted in the actual building modeled in our simulations, using the same set of six target objects. We deployed the Boston Dynamics Spot robot equipped with additional sensors and equipment, as described in Section 3.3.1.

The experiment followed a protocol similar to Simulated Experiment 1 in terms of target inspection sequence and presence of unmodeled static obstacles, though without the particularly challenging dynamic elements introduced in the simulations. Still, the building naturally contained obstacles and furniture that are not represented in the BIM model, as well as moving humans. This real-world experiment was intended to provide evidence that the navigation system functions correctly on physical hardware and can handle sensor noise, locomotion challenges, and environmental uncertainties difficult to replicate in simulations.

3.7.3. Data collection and metrics

For the simulated experiments, we recorded ROS2 bag files containing all sensor data, navigation commands, and internal system states. The system automatically logged images and associated metadata for each target, including position, orientation, and whether additional viewpoints were requested. For the real-world experiment, we largely followed the same methodology, but limited the collection of sensor

data and some internal states due to wireless bandwidth limitations. Additionally, we captured video recordings of the experiment from both the robot's camera and external cameras. The results from all experiments are discussed in Section 4.

We assessed the system's performance by visualizing the paths taken by the robot, the captured images, and the following quantitative metrics:

Navigation metrics. The following metrics are reported as the mean value per trial:

- **Path Length:** The total distance traveled by the robot.
- **Trial Duration:** The total time elapsed from mission start to finish.
- **Movement Speed:** The effective speed, calculated from the robot's first movement to its last (threshold > 0.1 m/s).

Task success metrics.

- **Execution Success Rate:** The proportion of targets for which the robot successfully reached a planned viewpoint and acquired an image. This measures the reliability of the navigation and image-capturing pipeline.
- **Qualitative Success Rate:** Of the successfully captured images (i.e., those counted in Execution Success Rate), this is the percentage deemed effective for inspection by a human evaluator. A success was recorded if the image clearly showed the target (or its empty location) without significant occlusion and with sufficient detail and quality to assess its condition. This metric specifically evaluates the performance of the NBV planner in selecting useful viewpoints.

VLM performance metrics. We evaluated the VLM's performance on two distinct classification tasks, calculated exclusively on images that passed the *Qualitative Success Rate* criterion. For each task, we report the accuracy, precision, recall, and the F1 score. The two tasks were:

- **Object Presence Detection:** Classifying whether the target object is present in the captured image.

Table 1

Quantitative performance metrics across three scenarios: Simulated Experiment 1 (Dynamic Environment, $N = 20$ runs), Simulated Experiment 2 (BIM Discrepancies, $N = 20$ runs), and Real-World Validation (single trial with 6 target inspections). Values show means with 95% confidence intervals in brackets.

Metric	Sim. Experiment 1	Sim. Experiment 2	Real Experiment 1
<i>Navigation Metrics</i>			
Path Length (m)	278.66 [243.10, 314.22]	331.80 [287.24, 376.35]	177.51
Trial Duration (s)	521.97 [460.61, 583.34]	564.27 [494.39, 634.16]	330.05
Movement Speed (m/s)	0.54 [0.51, 0.56]	0.59 [0.57, 0.61]	0.48
<i>Task Success Metrics</i>			
Execution Success Rate (%)	97.50 [92.91, 99.15]	100.0 [96.90, 100.0]	100.00
Qualitative Success Rate (%)	96.58 [91.54, 98.66]	96.67 [91.74, 98.70]	100.00
<i>VLM Performance (Object Presence)</i>			
VLM Accuracy (%)	77.78 [69.43, 84.36]	78.63 [70.36, 85.09]	100.00
VLM Precision (%)	98.63 [92.64, 99.76]	100.00 [94.93, 100.00]	100.00
VLM Recall (%)	74.23 [64.72, 81.89]	74.23 [64.72, 81.89]	100.00
VLM F1 Score (%)	84.71	85.21	100.00
<i>VLM Performance (Installation Status)</i>			
VLM Accuracy (%)	78.63 [70.36, 85.09]	80.34 [72.23, 86.53]	100.00
VLM Precision (%)	98.65 [92.73, 99.76]	100.00 [95.06, 100.00]	100.00
VLM Recall (%)	75.26 [65.82, 82.77]	76.29 [66.93, 83.65]	100.00
VLM F1 Score (%)	85.38	86.55	100.00

- **Installation Status Classification:** Determining the installation status of the target object, which could be inferred even if the object was not fully visible.

We present the metrics in Table 1. In the case of simulated experiments, we used two distinct methods for calculating the reported values and their 95% confidence intervals (CIs). For continuous metrics (path length, speed, and duration), the reported value is the mean of the 20 trial results, with the CI determined using the t-distribution. For proportion-based metrics (success rates and accuracies), all individual outcomes from all 20 trials were pooled to calculate a single aggregate success rate, with the CI for this aggregate proportion determined using the Wilson score method. For the real-world validation, which consisted of a single trial, all metrics are direct calculations without averaging or confidence intervals.

4. Results

Our autonomous navigation system demonstrated consistent performance in both simulated experiments and the real-world experiment. Table 1 summarizes the quantitative results from all experiments, showing high success rates for image capture despite the challenging dynamic obstacles and BIM discrepancies. The system effectively navigated to target locations and captured usable images of most objects, with execution success rates of 97.50% and 100% in the two simulation experiments and a complete success in the real-world validation.

In Simulated Experiment 2, the system was able to adapt to discrepancies between the BIM model and the actual environment. It successfully found paths through areas marked as walls in the BIM model, but that were absent in the simulated environment. This provides evidence for the effectiveness of our proposed cost-aware navigation approach, which assigns different traversal costs to verified and unverified BIM elements.

4.1. Experimental results

Simulated experiment 1: Dynamic environment. In the first simulation experiment, the system navigated through an environment with randomly placed obstacles and continuously moving virtual personnel, testing its ability to handle dynamic conditions. Fig. 10 illustrates how the robot navigated effectively toward the target locations despite the challenging environment. The figure depicts the initial 2D map derived from the BIM model in black, with blue circles marking the starting

positions for each trial. All trajectories taken across the trials are shown, along with small orange circles indicating the positions where images were captured; lines extending from these circles show the camera's direction of view. Red markings denote obstacles, such as closed doors, that were actually present in the simulated environment but not known to the system beforehand. Fig. 12 shows the highest quality images captured across different trials, with some variation in quality based on obstacle positions.

The timed cost decay feature of obstacles not defined in the BIM-model allows the robot to adapt as obstacles change position. Without this feature, the accumulated obstacle observations would likely have rendered large portions of the map inaccessible over time. Instead, the system continued working efficiently by reducing the cost of previously observed obstacles after they had not been observed for a while.

We consider it important to avoid triggering replanning instantly when lower-cost paths become available to prevent oscillation between potential paths. Instead, we trigger replanning only when the path has lethal obstacles. We theorize that this contributed to high execution success rate despite the challenging environment.

Beyond navigation performance, we evaluated the ability of the VLM to determine object status from the captured images. In our ground truth, all objects except the electrical door were present and installed. The electrical door was represented as a wall without a door, making “not present” and “not installed” the correct classifications for this target.

The system achieved an Execution Success Rate of 97.50%, meaning the robot successfully navigated to planned viewpoints and captured images for 97.50% of targets. Of these successfully captured images, 96.58% were deemed qualitatively useful for inspection purposes (Qualitative Success Rate). Finally, the VLM achieved 77.78% accuracy in detecting object presence and 78.63% accuracy in determining installation status when evaluated on this set of successfully captured images.

However, performance varied significantly by object type, as detailed in Table 3. The window (100% accuracy) and the electrical box (95% accuracy) were reliably identified as present, with matching accuracy for installation detection. Similarly, the electrical door was also reliably classified as absent (95% accuracy), demonstrating the VLM's ability to verify missing elements against BIM specifications. By contrast, the reception desk presented considerable challenges, with the desk achieving only 15% accuracy across both metrics. This performance disparity correlates directly with visual fidelity in the simulation: objects with simplified representations (the reception desk as an

abstract gray box and fire extinguishers as simple orange-red cylinders) were more frequently misidentified than objects with more realistic appearances. In a similar vein, one of the fire extinguishers (e) had noticeably lower success rates than the other two fire extinguishers. We speculate that this is due to the images of this extinguisher featuring more realistic background elements that overshadowed the simplified representation of the fire extinguisher.

Simulated experiment 2: BIM-reality discrepancy. The second simulation experiment tested the system's ability to navigate in environments with significant discrepancies between the BIM model and reality. As shown in Fig. 11, the environment was configured such that the only viable path between certain targets required traversing through a location where the BIM indicated a wall, but none existed in the simulation. The figure uses the same visual conventions as Fig. 10, showing how the system adapts to significant differences between the on-site environment and the BIM prior. The captured images are shown in Fig. 13.

Notably, the system achieved a 100% success rate in finding this viable path across all 20 trials. The navigation approach worked through a process of elimination: the robot would first attempt more obvious routes, encounter and verify BIM-defined walls that actually existed, mark them as lethal obstacles, and then explore alternatives. Eventually this led to identifying and utilizing the path through the non-existent BIM wall.

The results of this experiment provide evidence that our proposed system is capable of adapting to situations where BIM models differ from on-site conditions. By assigning non-lethal costs to unverified BIM elements, the system could still plan paths through them when necessary, while still avoiding BIM structures and prioritizing verified paths when available. The costmap at the beginning and end of one trial is depicted in Fig. 9, showing how the system removes BIM-defined walls and adds obstacles in the map during the experiment.

The system achieved an Execution Success Rate of 100%, with all targets successfully reached and imaged. Of these captured images, 96.67% were deemed qualitatively useful for inspection purposes (Qualitative Success Rate). The VLM performed comparably to Experiment 1, achieving 78.63% accuracy in detecting object presence and 80.34% accuracy in determining installation status when evaluated on this set of successfully captured images. The ground truth remained the same: all objects except the electrical door were present and installed. Object-specific performance patterns remained similar, with the window and electrical door achieving perfect accuracy rates (100%), while the reception desk continued to present significant challenges with only 10% presence accuracy. Similarly to Simulated Experiment 1, the same fire extinguisher (e) underperformed relative to the others.

Practical experiment. The real-world validation provided evidence that the system translates to physical hardware in an actual building environment. As shown in Fig. 14, the system effectively navigated through the building and reached all target positions. Fig. 15 presents the images captured during this experiment, showing successful capture of all targets that were visible from the navigable space and installed as specified in BIM.

A notable challenge encountered in the real-world deployment was that two fire extinguishers were installed at locations that differed from their BIM specifications. Fire extinguisher 1 was close enough to be visible in the captured image (Fig. 15), but fire extinguisher 2 was installed on the opposite side of a structural column from its defined position, making it impossible to capture from the planned viewpoint (Fig. 15). In this case, the robot correctly navigated to the planned viewpoint and captured an image that accurately showed the target was not present in its specified location. According to our evaluation criteria, this was considered a successful capture, as the system provided a clear view of the designated inspection area, even though the object itself was not visible. This highlights a limitation of our current approach, which relies on accurate BIM positioning data



(a) 2D costmap projection of the 3D voxel map at the start of one trial showing cells with occupancy probability > 50%.



(b) Similar view at the end of the same trial. The robot has added obstacles to the map and removed walls that were initially defined in the BIM but have not yet been constructed or are otherwise missing.

Fig. 9. Comparison of occupancy maps before and after the robot's modifications in a single trial (Simulated Experiment 2).

for target objects and does not incorporate active search capabilities for misplaced objects.

We also identified a hardware limitation related to LiDAR visibility, as the robot's arm partially obstructed forward-facing sensor readings. This limitation was mitigated by the Spot robot's native obstacle avoidance capabilities using its own forward-facing depth sensors, which complemented our navigation approach.

Fig. 16 shows the robot during the experiment as it inspects one of the fire extinguishers. The sequential visiting and image capture of targets proceeded as designed, validating the overall workflow in physical deployment.

For this real-world experiment, we evaluated the VLM's performance with a more nuanced ground truth that accounted for real-world installation discrepancies. We considered the electrical door, fire extinguisher 1, reception desk, and fire extinguisher 3 as present and installed (although fire extinguisher 1 was not at its exact BIM position). Fire extinguisher 2 was considered not present and not installed from our inspection perspective, as it was installed on the opposite side of a structural column from its BIM-specified location and thus not visible from the planned viewpoint. For the large window, we established an interesting ground truth distinction: not present in the image (as it was fully obscured by curtains) but installed (as the presence of curtains implicitly indicates a window installation).

The VLM achieved 100% accuracy in determining installation status against this real-world ground truth. Its performance on the obscured window was particularly noteworthy. Despite not directly observing the

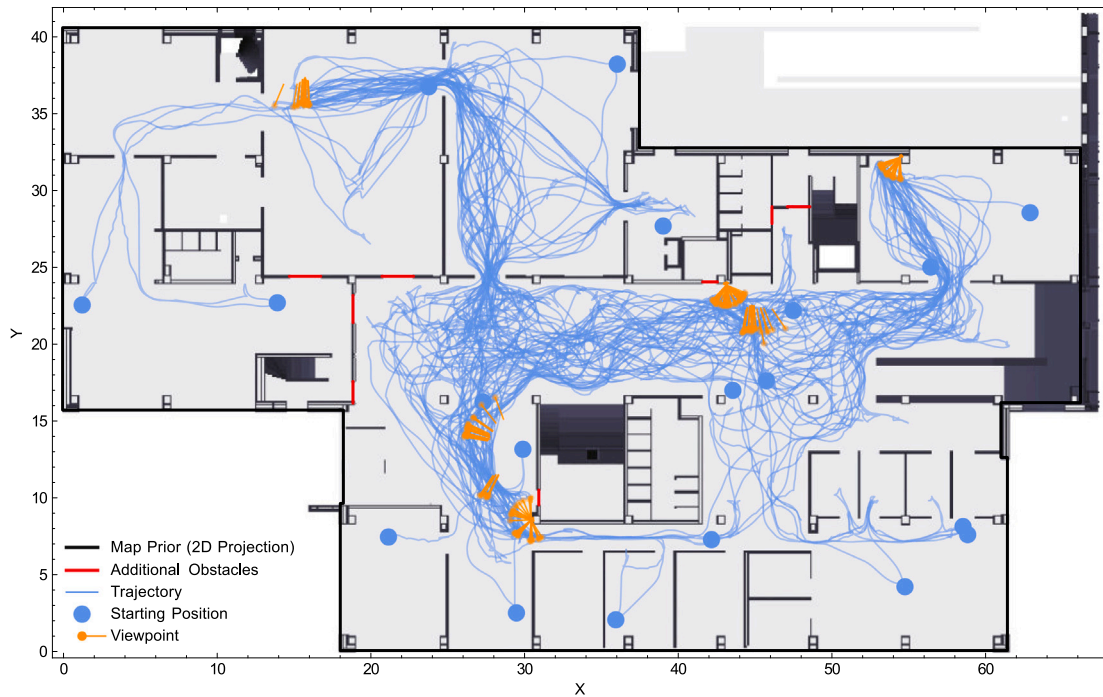


Fig. 10. Map of Simulated Experiment 1 showing starting positions, trajectories, and image capture positions across all trials.

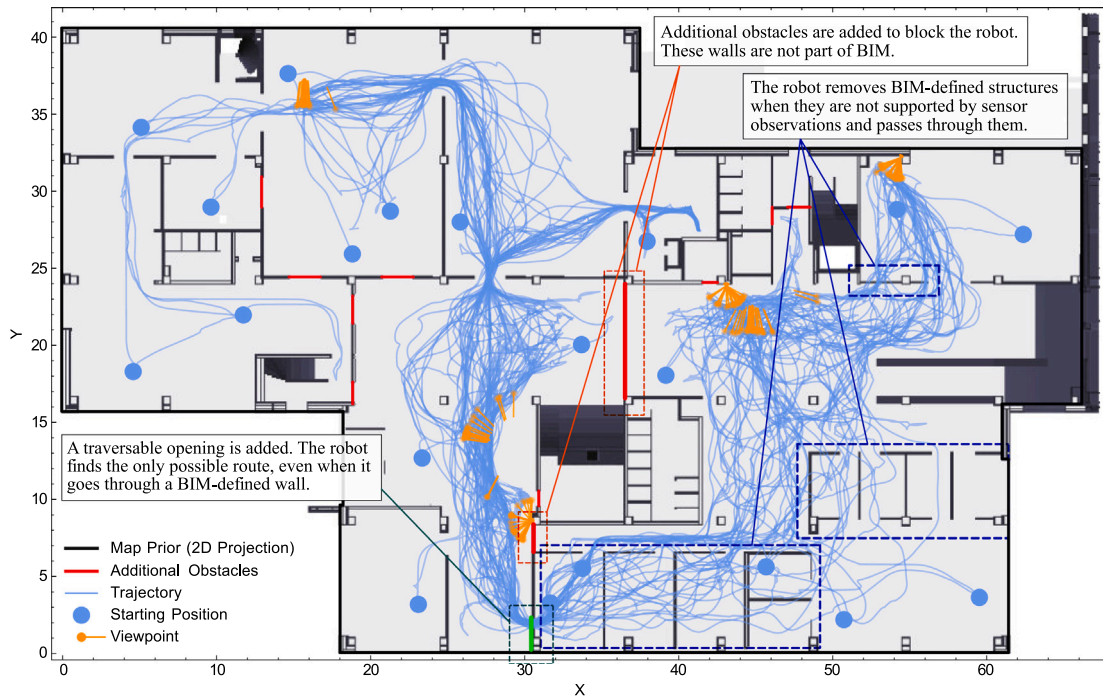


Fig. 11. Map of Simulated Experiment 2 showing how the system adapts to significant differences between the on-site environment and the BIM prior.

window behind the curtains, the VLM correctly inferred its installation status (installed). Importantly, it provided a rationale in the generated reasoning, explaining that while the window itself was not visible, the presence of curtains strongly indicates that a window is installed. This ability to perform logical inference from contextual cues, combining object recognition (curtains) with architectural knowledge, highlights a potential advantage of using VLMs over traditional visual object detectors for complex inspection tasks. However, the strong VLM performance in this experiment should be interpreted with great caution due to the small sample size.

4.2. Comparative analysis

4.2.1. Simulation vs. real-world performance

Path planning, obstacle avoidance, and BIM integration functioned similarly across both simulations and the real-world validation, indicating that our simulation environment functions as a viable testbed for system development.

A key difference in the quantitative results (Table 1) was the significantly shorter path length (177.51 m) and trial duration (330.05 s) in the real-world experiment compared to Simulated Experiment 1

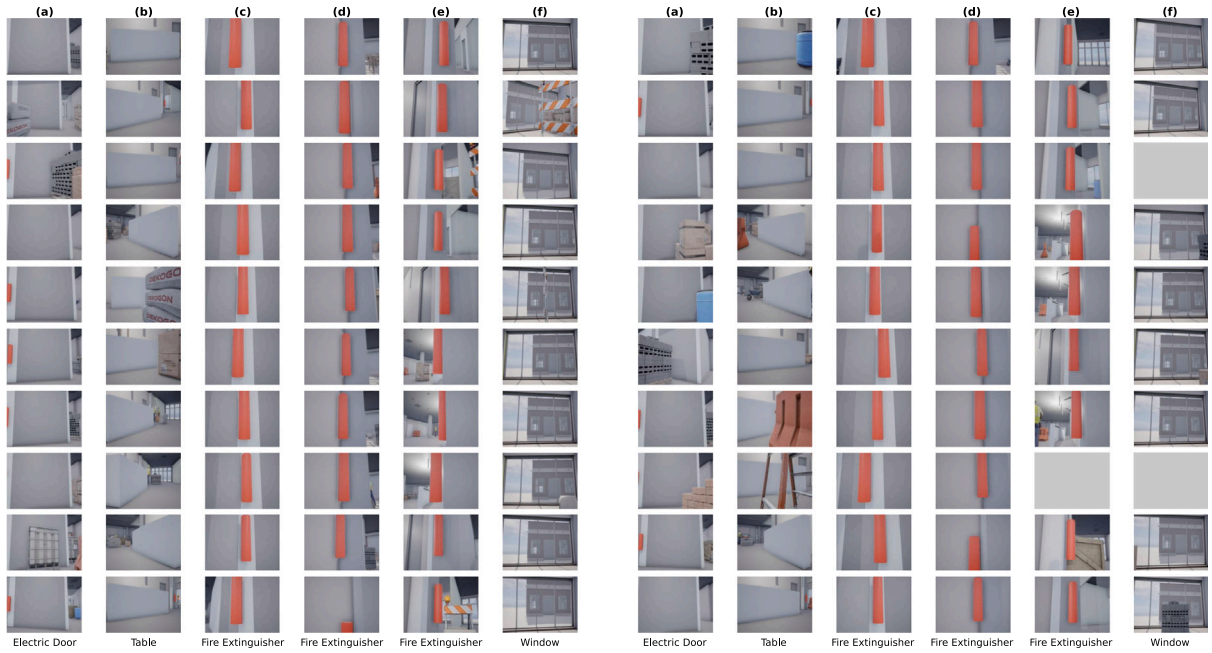


Fig. 12. Captured images from Simulated Experiment 1, showcasing the highest quality image of each target object as calculated by the NBV scoring function. Each row represents a different trial, while each column corresponds to a different target object. Gray 'blank' images indicate instances where the system failed to capture an image of the respective target object.

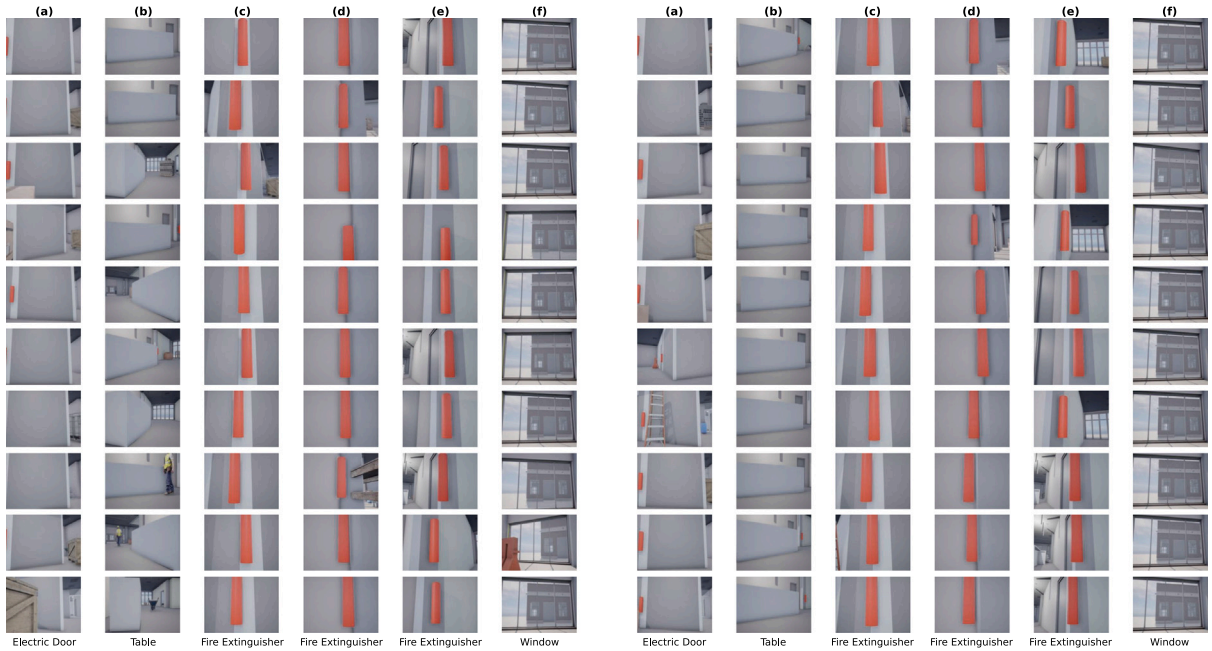


Fig. 13. Captured images from Simulated Experiment 2, showcasing the highest quality image of each target object as calculated by the NBV scoring function. Each row represents a different trial, while each column corresponds to a different target object.

(278.66 m and 521.97 s). This discrepancy highlights the intentionally challenging simulation, which included 80 randomly placed obstacles and 10 continuously moving agents, forcing frequent re-planning and longer detours. In contrast, the real-world environment, while containing unmodeled obstacles, was less cluttered, making navigation easier. Still, these real-world obstacles not represented in the BIM (furniture, temporary materials) were successfully integrated into the 3D voxel

map, and by extension, the navigation costmap, allowing the robot to plan appropriate paths around them. This demonstrates the system's transferability to real-world settings and its ability to handle BIM-reality discrepancies, although the real-world discrepancies were less severe than those in Simulated Experiment 2.

While the single real-world trial provides valuable validation, it does not offer the statistical confidence of simulation experiments.

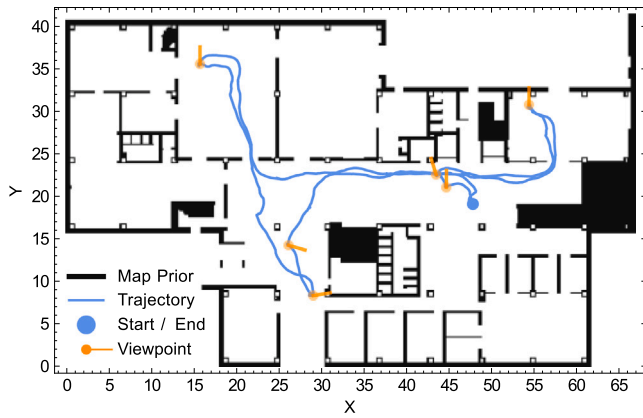


Fig. 14. Map showing the trajectory and viewpoints taken by the robot during the practical validation experiment.

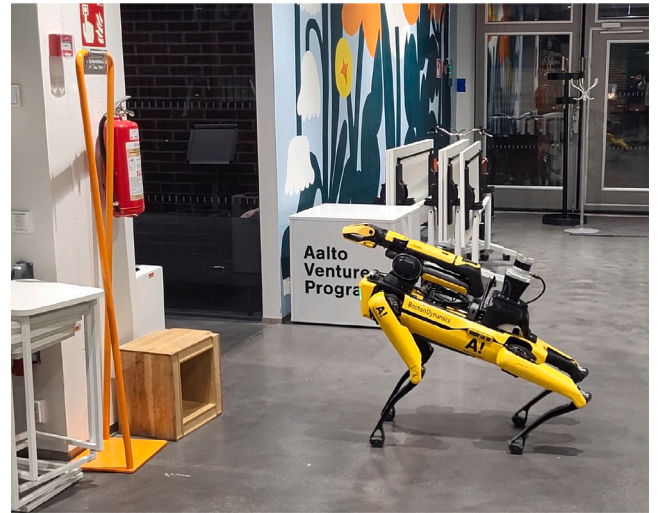


Fig. 16. Boston Dynamics Spot robot inspecting a fire extinguisher.

This cautious behavior appears to correlate with the visual fidelity of simulated objects. For instance, the reception desk, rendered as a simple gray box, achieved presence accuracy scores of only 15% and 10% in the two simulations. This poor performance is understandable, as the object lacked distinguishing features (shown in Figs. 12 and 13). In contrast, the visually distinct window was identified with near perfect accuracy. The fire extinguishers also highlight this sensitivity. While most were identified with 95% accuracy, extinguisher (e) consistently underperformed, likely due to more varied backgrounds in its viewpoints. This indicates that VLM performance is highly dependent on the quality and distinctiveness of visual evidence.

By contrast, in the real-world experiment (six images), the VLM confidently and correctly classified all objects. The model demonstrated more confident reasoning when provided with visually richer images, including making a logical leap regarding the window installation behind curtains. When presented with the same reception desk object type in the real-world context (shown in Fig. 15 as object (b)), the VLM identified not only the desk's form but also noted the presence of "Aalto School of Engineering" signage behind it, using this contextual information to support its classification. The ability of the VLM to combine object detection with text recognition and contextual inference represents a capability that is beyond conventional object detectors, particularly for the discrete object types evaluated in our experiments. Furthermore, the model's use of textual rationales, as seen in these examples, provides a potential layer of traceability for understanding its decisions.

4.2.3. System variant comparisons

To evaluate the contributions of our system's key components, we performed comparative experiments against two baseline variants. These experiments were conducted with $N = 10$ trials each, a number we determined to be sufficient for demonstrating meaningful performance differences. The *Non-Semantic* variant removes our custom BIM-aware map update and cost assignment logic (Sections 3.4.2 and 3.4.3), reverting to a standard voxel map with binary occupancy. The *No NBV Retries* variant removes adaptive viewpoint refinement, relying on a single set of viewpoints calculated at initialization.

As shown in Table 2, both variants exhibit shorter path lengths and experiment durations. However, this apparent efficiency is misleading, as it stems from frequent mission failures where targets are not reached. This difficulty can be reflected in their lower movement speeds, which point to hesitant navigation with frequent replanning and course corrections.

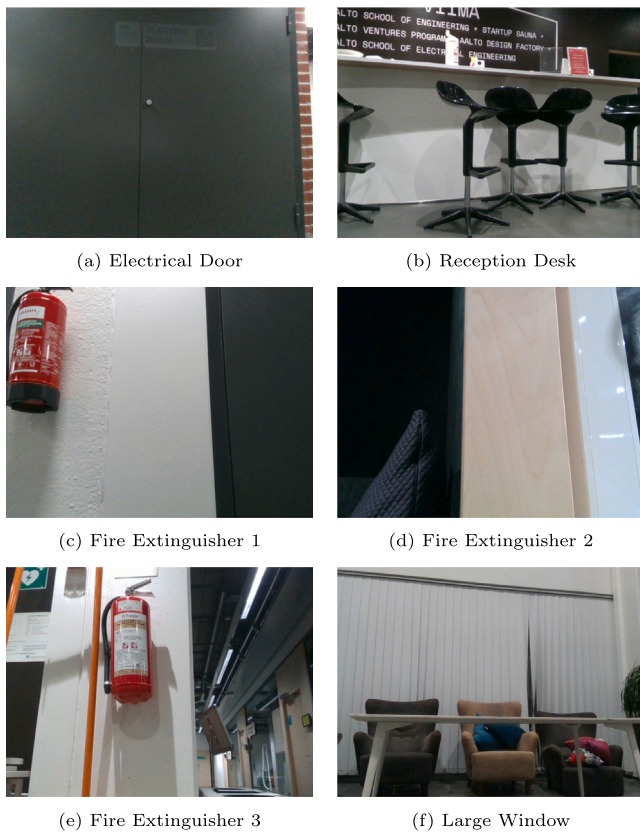


Fig. 15. Collection of RealSense D415 images from the validation experiment.

Nevertheless, the successful execution of the complete workflow in a physical environment supports the practical viability of our approach.

4.2.2. VLM performance analysis

The performance of the VLM in verifying object presence and installation status is detailed in Table 1, which presents the accuracy, precision, recall, and F1 scores.

The quantitative results for both simulated environments show a pattern that merits analysis: precision of close to 100% coupled with recall of $\approx 74\%$. This combination suggests that the VLM operated with considerable caution. The high precision indicates that the system rarely produced false positives, while the lower recall reveals that the model frequently failed to confirm the presence of correctly placed objects, producing false negatives instead.

Table 2
Simulated comparison of system variants.

Metric	Ours	Non-Semantic	No NBV Retries
<i>Execution Success Rate (%)</i>			
Simulated Experiment 1	97.50 [92.91, 99.15]	81.67 [70.08, 89.44]	80.00 [68.22, 88.17]
Simulated Experiment 2	100.00 [96.90, 100.0]	55.00 [42.49, 66.91]	76.67 [64.56, 85.56]
<i>Path Length (m)</i>			
Simulated Experiment 1	278.66 [243.10, 314.22]	190.73 [139.29, 242.17]	226.63 [178.20, 275.06]
Simulated Experiment 2	331.80 [287.24, 376.35]	142.00 [97.17, 186.83]	258.79 [164.72, 352.87]
<i>Trial Duration (s)</i>			
Simulated Experiment 1	521.97 [460.61, 583.34]	377.25 [287.06, 467.43]	404.40 [323.19, 485.60]
Simulated Experiment 2	564.27 [494.39, 634.16]	302.34 [235.73, 368.94]	463.89 [305.53, 622.24]
<i>Movement Speed (m/s)</i>			
Simulated Experiment 1	0.54 [0.51, 0.56]	0.50 [0.44, 0.57]	0.56 [0.54, 0.59]
Simulated Experiment 2	0.59 [0.57, 0.61]	0.46 [0.39, 0.53]	0.55 [0.51, 0.60]

Table 3

Per-object VLM accuracy for presence detection and installation status assessment across simulated experiments. Values show accuracy percentages with 95% confidence intervals in brackets. Abbreviations: Elec. = Electronic, Recep. = Reception, Ext. = Extinguisher.

Object	Sim. Exp. 1	Sim. Exp. 2
<i>VLM Performance (Object Presence)</i>		
(a) Elec. Door	95.00 [76.39, 99.11]	100.00 [83.89, 100.00]
(b) Recep. Desk	15.00 [5.24, 36.04]	10.00 [3.01, 30.10]
(c) Fire Ext.	95.00 [76.39, 99.11]	95.00 [76.39, 99.11]
(d) Fire Ext.	95.00 [76.39, 99.11]	95.00 [76.39, 99.11]
(e) Fire Ext.	68.42 [46.04, 84.64]	73.68 [51.21, 88.19]
(f) Window	100.00 [82.41, 100.00]	100.00 [82.41, 100.00]
<i>VLM Performance (Installation Status)</i>		
(a) Elec. Door	95.00 [76.39, 99.11]	100.00 [83.89, 100.00]
(b) Recep. Desk	15.00 [5.24, 36.04]	15.00 [5.24, 36.04]
(c) Fire Ext.	95.00 [76.39, 99.11]	95.00 [76.39, 99.11]
(d) Fire Ext.	95.00 [76.39, 99.11]	100.00 [83.89, 100.00]
(e) Fire Ext.	73.68 [51.21, 88.19]	73.68 [51.21, 88.19]
(f) Window	100.00 [82.41, 100.00]	100.00 [82.41, 100.00]

The impact of our contributions is most evident in the Execution Success Rates. The Non-Semantic variant's performance collapses in the BIM discrepancy scenario (Experiment 2), dropping from our system's 100% success to 55.00%. Lacking an adaptive verification mechanism, it treats the initial BIM as ground truth, preventing it from navigating through areas where walls were defined but did not physically exist in reality.

This variant's handling of dynamic obstacles is equally problematic. Any detected obstacle is marked as a permanent, lethal obstruction. A path temporarily blocked by a moving agent remains impassable on the map long after the agent has moved on. Over the course of a mission, the map accumulates these "ghost" obstacles, leading the robot to incorrectly perceive valid pathways as permanently blocked. This forces it to either find significantly longer alternative routes or, more often, conclude that a target is unreachable, directly causing mission failures and contributing to the reduced success rates in both experiments.

Similarly, the No NBV Retries variant highlights the necessity of adaptive viewpoint planning in dynamic settings. Its static, pre-calculated viewpoints were often obstructed by dynamic obstacles, leading to failed capture attempts (failure to navigate to the pre-planned viewpoint). It is important to note that our execution success rate metric measures task completion (reaching a viewpoint), not the quality of the resulting image. Therefore, the true degradation in inspection effectiveness for this variant is likely even more significant than the metric suggests.

5. Contribution to knowledge

This research advances robotic navigation and inspection in dynamic built environments through several methodological and engineering contributions. Our framework integrates BIM geometry, textual metadata, and LiDAR/visual sensor data, enabling a quadruped robot to perform tasks that require a synthesis of structural, semantic, and real-world information.

First, our localization strategy effectively bridges BIM data with real-time sensor measurements. By initializing 3D SLAM with BIM geometry, we maintain consistent alignment between sensor observations and the BIM coordinate frame, enabling reliable operation despite floor-level clutter and missing structures. The successful navigation results across environments with BIM-reality discrepancies demonstrate the approach's effectiveness in handling environmental challenges.

Second, our adaptive mapping and navigation approach addresses a critical challenge in robotic operation within these dynamic built environments. The system distinguishes between verified structures, unbuilt BIM elements, and temporary obstacles, enabling navigation through areas marked as walls in BIM but not yet built, while avoiding confirmed structures. This improves navigation reliability in environments where BIM models differ from on-site reality.

Third, we advance object inspection capabilities by integrating next-best-view (NBV) planning with our 3D mapping approach that synthesizes BIM geometry and sensor data. This enables the system to select effective viewpoints that account for both designed geometry and real-world conditions, improving inspection coverage.

Fourth, our integration of vision-language models (VLMs) within a robotic inspection workflow demonstrates how foundational AI models can be applied to domain-specific construction tasks. Our approach enables cross-modal semantic integration by combining structural BIM metadata with visual observations and language understanding to automatically determine the presence and installation status of target objects. By using BIM information as a semantic reference, we eliminate the need for task-specific model training, showing how general-purpose language understanding can be leveraged to serve construction object inspection applications.

Collectively, these contributions deliver an end-to-end framework for robotic inspection. The key insight underlying our approach is that treating BIM as probabilistic prior knowledge, rather than ground truth, enables robots to adapt to the uncertainty of construction and built environments. The framework fuses structural BIM data with real-time sensing to handle discrepancies between design models and site reality, while incorporating language-based reasoning for semantic object understanding and status determination. Our real-world deployment on a quadruped robot suggests the technical feasibility of this integrated approach. This represents tangible progress toward deployable systems that can handle the characteristic challenges of built environments: dynamic changes, design-reality discrepancies, and unexpected obstacles.

6. Limitations and future work

BIM integration and quality. Our system's performance is fundamentally dependent on the input BIM model accuracy and quality. The system requires BIM models with Level of Development (LoD) 300 and above, where key elements are geometrically accurate and carry sufficient semantic information for inspection tasks.

While our framework can theoretically handle various BIM-reality discrepancies such as added obstacles and missing walls, the boundaries of tolerable inconsistencies remain unclear. We assume that SLAM localization would fail before other system components when discrepancies become excessive, though systematic evaluation of these failure thresholds has not been conducted. Additionally, greater BIM discrepancies require more accurate SLAM initialization to maintain system performance.

Real-world validation revealed a dependency on BIM positional accuracy when fire extinguishers installed at locations different from BIM specifications could only be detected if they remained visible from the planned camera viewpoints. This highlights our lack of active search capabilities for misplaced objects, though it is debatable whether objects installed contrary to BIM specifications constitute system failure or specification non-compliance.

Additionally, our approach primarily relies on static as-designed BIM models without integrating dynamic external data sources that could provide real-time updates about evolving site conditions.

Future work should systematically evaluate system robustness against various BIM quality issues to establish clear tolerance boundaries and failure modes. Research should also explore active search behaviors for misplaced objects and expand real-time BIM validation using VLMs to all encountered objects, not only inspection targets. Building on our current dynamic map updating capabilities, a particularly significant avenue for future work is to enable a bidirectional information flow by propagating these corrections back to update the source BIM model itself. This would enable broader project teams to benefit from observed site conditions. Furthermore, our parallel research is investigating the use of semantic digital twin concepts [70] to fuse structured construction data with ontology-based reasoning, incorporating inputs from on-site sensors to enrich the model with real-time positioning and environmental data for more adaptive robotic task planning that can better respond to evolving site conditions and changing project requirements.

VLM capabilities. Our vision-language approach faces several significant limitations that constrain installation verification accuracy. First, our VLM-based verification works well for components completed in a single stage but struggles with multi-stage construction elements. Many building components undergo intermediate installation phases not represented in BIM models, such as frames being installed before panels, followed by finishing and painting. The current approach can only distinguish between simple installed/not-installed classifications rather than recognizing progressive construction stages.

Second, the VLM's effectiveness appears highly dependent on the object type due to fundamental limitations in VLM spatial and instance-level reasoning. This is particularly challenging for continuous structural elements like walls or slabs. While the VLM can determine if an image contains "a wall", it lacks the precise geometric grounding to verify if the specific wall segment at a given location is the one in question. This could lead to false positives where the model observes a nearby, existing wall and incorrectly confirms the installation of a new, missing segment. A related challenge arises when differentiating between multiple, visually identical instances of discrete objects (e.g., two fire extinguishers) within a single view. In such cases, the VLM can confirm the presence of an object type but cannot reliably link it to a specific BIM entity, creating ambiguity. Therefore, the current approach is likely most suitable for visually distinct, discrete objects that can be uniquely identified within an image.

Third, VLM performance has not been rigorously evaluated in real-world scenarios. While our simulation results showed possible correlations between accuracy rates and visual fidelity, with simplified object representations frequently misidentified compared to more distinctively rendered objects, many failures appeared to stem from low visual fidelity rather than actual VLM limitations. Our limited real-world experiment showed perfect VLM performance, but this single trial is insufficient for comprehensive performance assessment.

Finally, while the VLM provides a textual rationale for its decisions, offering a degree of traceability, there is no structured mechanism to automatically parse these explanations or quantify the model's confidence. Instead, the system works with binary classifications without a clear mechanism to understand why the VLM made specific decisions. This largely leaves the interpretation of unexpected results to human review of the captured images or the generated textual reasoning. This creates a black box process that limits interpretability, a common challenge in deep-learning computer vision approaches.

Future research should explore appearance-based methods that can recognize and track progressive construction stages beyond binary installation classifications and improve spatial reasoning capabilities for precise component-level tracking, potentially by integrating VLMs with geometric data, such as 3D point clouds. Additionally, enhancing VLM interpretability would improve system transparency. Most critically, meaningful failure analysis requires extensive testing on large real-world datasets to properly evaluate VLM performance under realistic construction conditions.

Navigation and environmental constraints. Our system faces several limitations related to robot navigation and environmental operation. First, the system is designed for structured environments with architectural features, as our SLAM system relies on these elements (walls, ceiling features) for reliable positioning with respect to BIM. Performance degrades when structural features become heavily occluded by obstacles or personnel, potentially leading to localization failures.

Second, the SLAM system requires manual initialization of the robot's approximate pose relative to the BIM coordinate frame. While the required precision is environment-dependent and difficult to quantify, initialization success can be immediately verified through LiDAR-BIM alignment visualization. Automatic global localization would be preferable, but geometric symmetry in construction environments makes this challenging with LiDAR alone.

Third, our NBV planning component applies a 0.3-meter margin around target objects to address localization and sensor inaccuracies, excluding voxels within this margin from collision checks. While this prevents misclassifying target-adjacent structures as obstacles, it creates the risk of missing actual obstacles very close to targets, potentially preventing proper camera observation of target objects.

Fourth, our implementation lacks sophisticated human-robot interaction capabilities. While the system avoids humans as it avoids all obstacles, it does not incorporate specific safety protocols or communicate intent to workers. Our current navigation approach may be insufficient for truly crowded construction environments with multiple dynamic obstacles and workers.

Fifth, our system faces inherent sensing limitations common to ground robots. The platform has limited ability to capture elements positioned above the robot, constraining inspection capabilities for certain types of building components, particularly ceiling-mounted elements. Additionally, although several system components operate three-dimensionally, our navigation planning currently handles only single-floor operations.

Sixth, the system lacks explicit degradation strategies for sensor failures such as LiDAR occlusion. When sensor data becomes compromised, the system continues normal operation relying solely on the inherent robustness of the underlying SLAM algorithm without active failure detection or response mechanisms.

Finally, the system has not addressed multi-robot collaboration, meaning robots operate independently without coordinating paths, sharing tasks, or managing potential conflicts among multiple units.

Future research should extend path planning to true 3D navigation beyond single-floor scenarios, developing coordination protocols with smart building systems (such as elevators) for autonomous navigation between floors and handling vertical movement on structures like stairs. Enhancing the system with predictive human movement modeling, and clear visual/audio signaling would significantly improve operational safety in crowded environments. Additionally, developing active failure detection and graceful degradation strategies would improve system robustness when sensors are compromised. From a system architecture perspective, future work could also address the current representational redundancy between separate SLAM maps, 3D voxel maps, and 2D costmaps, aiming for a unified representation to improve efficiency and robustness. For example, the semantic mapping approach could be utilized to improve SLAM by weighting landmarks based on their permanence and source. Finally, multi-robot deployment would benefit from cooperative planning, communication, and collision avoidance.

System evaluation and validation. Our experimental validation faces two significant limitations that affect the robustness of our performance assessment. First, our real-world validation was limited to a single trial, providing practical proof of concept but lacking the statistical robustness of our simulation experiments. This limitation was primarily due to the practical challenges and resource requirements of conducting multiple real-world trials in a controlled building environment.

Second, our evaluation focused on task-level performance metrics rather than measuring component-level localization errors. While high execution success rates across multiple trials suggest that localization accuracy was sufficient for task demands, we lack direct localization error metrics that would strengthen the technical rigor of our evaluation. More subtle localization inaccuracies could affect system performance aspects such as how precisely the robot positions itself relative to BIM-defined viewpoints during image capture, effects that would be most relevant in real-world deployments where ground truth positioning data is unavailable.

Future work should conduct more extensive real-world experiments to enable thorough evaluation of the integrated system across diverse construction scenarios, including assessment of navigation efficiency, viewpoint planning effectiveness, and component recognition accuracy. Additionally, developing methods to assess localization accuracy in real-world deployments would provide better understanding of how localization precision affects different aspects of the inspection pipeline.

7. Conclusion

This paper addressed the challenge of deploying autonomous robots for object inspection in dynamic and complex environments by leveraging BIM. Despite considerable research on construction robotics, operating in partially built environments presents challenges due to discrepancies between as-designed and as-built conditions, dynamic obstacles, and frequent changes in the environment.

A critical finding from our integration effort is the importance of system-level design. The interdependencies between BIM processing, SLAM, mapping, and navigation are significant, and these components should be co-designed and tested together rather than independently optimized. The successful operation of the robot was not due to a single component, but rather the specific implementation choices that integrated these components, suggesting that the “blueprint” for a working system is itself a critical contribution: the manner in which existing techniques can be adapted and integrated to function together.

This interdependence is strongly demonstrated in a core principle of mobile robotics: navigation is not just one component among many, but the foundational capability upon which many higher-level tasks are

built. Therefore, the design of related components cannot be considered in isolation from how they enable or inhibit robust navigation. This became evident during development, where the adaptations to the mapping system, for example, were not abstract improvements but practical changes driven almost entirely by the need to achieve robust navigation in the target environmental conditions.

Construction robotic systems in particular are built around as-designed models, which makes robust BIM integration particularly important. Systems that blindly follow static BIM models are efficient but inflexible when facing BIM-reality discrepancies, while systems that ignore BIM entirely are more adaptable but sacrifice the efficiency gains of a priori knowledge. Hybrid approaches arguably gain the benefits of both, as they can benefit from design information for efficient pathing while gracefully adapting when that information proves incorrect.

In practice, this requires recognizing that robots must handle multiple, potentially conflicting sources of environmental information. Our BIM-integrated navigation system addresses this through two key insights. First, different types of obstacles require different navigation strategies: our cost-aware approach assigns traversal costs based on obstacle source and on whether BIM-defined obstacles have been confirmed by sensor observations, enabling robots to navigate confidently around verified structures while exploring areas where BIM elements may be missing or not yet constructed. Second, temporary and permanent obstructions must be distinguished over time to prevent navigation maps from becoming permanently blocked by temporary obstacles.

We demonstrated the feasibility of our approach by integrating it into a workflow for automated robotic object inspection. The system utilizes the navigation capabilities to sequentially visit target objects defined in the BIM, capture appropriate images, and evaluate installation status. Our experiments, conducted across both simulated and real-world environments, demonstrate the system's effectiveness in navigating complex, partially built spaces. The navigation system successfully handled environments with numerous dynamic obstacles and significant BIM-reality discrepancies, consistently finding viable paths even when the only available route required traversing through areas marked as walls in the BIM model but not yet constructed in reality.

While the object inspection application utilizes established techniques for viewpoint planning and image capture, its distinct contribution lies in its successful integration of localization, mapping, and navigation into an autonomous, end-to-end system. A key element of this integrated approach is the verification of installed components. Traditional occupancy-based methods are vulnerable to discrepancies between as-designed and as-built object geometry or placement, while conventional object detection models face prohibitive data requirements for specialized construction components. By leveraging general-purpose vision-language models, our approach inherently performs the mapping from textual BIM metadata to a visual understanding of the target object, making inspection processes more tractable across the large range of components found on a construction site.

This work also highlights several clear avenues for future research. The system's integration could be significantly deepened to eliminate the current representational redundancy, where separate SLAM maps, 3D voxel maps, and 2D costmaps serve overlapping functions. A unified representation that maintains the semantic distinctions between obstacle types while serving all navigation and mapping purposes could improve efficiency and robustness. This deeper integration could also enable true 3D navigation, extending the cost-based obstacle classification beyond the current 2D path planning to handle tasks requiring vertical movement, such as navigating stairs or multi-level scaffolding. Additionally, the semantic mapping approach could be applied to improve the robustness of SLAM itself by weighting different obstacles based on their status and source. The VLM approach could be extended to distinguish multi-stage construction elements, while multi-robot collaboration could enable agents to share environmental observations for greater efficiency. Perhaps most significantly, the system could be extended to enable bidirectional BIM information flow. The system

already tracks and verifies the occupancy of structural elements like walls for navigation purposes, but this information is currently under-utilized. Future work could leverage these environmental observations to automatically update the BIM model.

In conclusion, this work presents a step toward autonomous robotic systems operating reliably in dynamic environments. The robust navigation capability demonstrated for inspection directly enables other tasks requiring reliable mobility in such environments, including automated 3D scanning of construction progress and post-construction facility monitoring, where systems benefit from following building models while having the ability to adapt to discrepancies. By presenting not only methodological contributions but also a complete and tested system, we provide a foundation for these and other applications requiring robust navigation. The demonstrated ability to handle BIM-reality discrepancies and dynamic obstacles addresses fundamental challenges in construction automation, ultimately supporting improved efficiency in construction processes.

CRedit authorship contribution statement

Jaakko Tuomisto: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Conceptualization. **Yuan Zheng:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Data curation, Conceptualization. **Inshu Chauhan:** Writing – review & editing, Writing – original draft, Conceptualization. **Olli Seppänen:** Writing – review & editing, Supervision, Conceptualization. **Joni Pajarinen:** Writing – review & editing, Supervision, Conceptualization.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Anthropic Claude and Google Gemini for grammar checking and limited language advice in specific sections. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Acknowledgments

We acknowledge the financial support of the Finnish Ministry of Education and Culture through the Intelligent Work Machines Doctoral Education Pilot Program (IWM VN/3137/2024-OKM-4).

This research was also supported by Research Council of Finland (357301) and the ACTOR project (7376/31/2021). ACTOR is supported by Business Finland's Low Carbon Built Environment Program, which receives funding from the EU's Recovery and Resilience Facility (until 2024).

Declaration of competing interest

The authors declare that they possess no financial conflicts of interest or personal connections that might reasonably be perceived as influencing the research findings presented in this manuscript.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.autcon.2025.106571>.

Data availability

Selected data from this study can be made available upon request.

References

- [1] Z. Pučko, N. Šuman, D. Rebolj, Automated continuous construction progress monitoring using multiple workplace real time 3D scans, *Adv. Eng. Inform.* 38 (2018) 27–40, <http://dx.doi.org/10.1016/j.aei.2018.06.001>.
- [2] F. Shalabi, Y. Turkan, IFC BIM-based facility management approach to optimize data collection for corrective maintenance, *J. Perform. Constr. Facil.* 31 (1) (2017) 04016081, [http://dx.doi.org/10.1061/\(ASCE\)CF.1943-5509.0000941](http://dx.doi.org/10.1061/(ASCE)CF.1943-5509.0000941).
- [3] M. Kopsida, B. Ioannis, P. Vela, A review of automated construction progress monitoring and inspection methods, in: *Proc. of the 32nd CIB W78 Conference*, 2015, Eindhoven, The Netherlands.
- [4] R. Samsami, A systematic review of automated construction inspection and progress monitoring (ACIPM): Applications, challenges, and future directions, *CivilEng* 5 (1) (2024) 265–287, <http://dx.doi.org/10.3390/civileng5010014>.
- [5] R. Grosso, U. Mecca, G. Moglia, F. Prizzon, M. Rebaudengo, Collecting built environment information using UAVs: Time and applicability in building inspection activities, *Sustainability* 12 (11) (2020) 4731, <http://dx.doi.org/10.3390/su12114731>.
- [6] G.P.M. Vassena, L. Perfetti, S. Comai, S. Mastrolembo Ventura, A.L.C. Ciribini, Construction progress monitoring through the integration of 4D BIM and SLAM-based mapping devices, *Buildings* 13 (10) (2023) 2488, <http://dx.doi.org/10.3390/buildings13102488>.
- [7] G. Gajamani, K. Varghese, Automated project schedule and inventory monitoring using RFID, in: *Automation and Robotics in Construction - Proceedings of the 24th International Symposium on Automation and Robotics in Construction*, 2007, pp. 47–53, <http://dx.doi.org/10.22260/ISARC2007/0012>.
- [8] J. Zhao, O. Seppänen, A. Peltokorpi, B. Badihi, H. Olivieri, Real-time resource tracking for analyzing value-adding time in construction, *Autom. Constr.* 104 (2019) 52–65, <http://dx.doi.org/10.1016/j.autcon.2019.04.003>.
- [9] P. Kavaliuskas, J.B. Fernandez, K. McGuinness, A. Jurelionis, Automation of construction progress monitoring by integrating 3D point cloud data with an IFC-based BIM model, *Buildings* 12 (10) (2022) 1754, <http://dx.doi.org/10.3390/buildings12101754>.
- [10] H. Omar, L. Mahdjoubi, G. Kheder, Towards an automated photogrammetry-based approach for monitoring and controlling construction site activities, *Comput. Ind.* 98 (2018) 172–182, <http://dx.doi.org/10.1016/j.compind.2018.03.012>.
- [11] B. Ekanayake, J.K.-W. Wong, A.A.F. Fini, P. Smith, Computer vision-based interior construction progress monitoring: A literature review and future research directions, *Autom. Constr.* 127 (2021) 103705, <http://dx.doi.org/10.1016/j.autcon.2021.103705>.
- [12] M. Sami Ur Rehman, M.T. Shafiq, F. Ullah, Automated computer vision-based construction progress monitoring: a systematic review, *Buildings* 12 (7) (2022) 1037, <http://dx.doi.org/10.3390/buildings12071037>.
- [13] J. Bohn, J. Teizer, Benefits and barriers of construction project monitoring using high-resolution automated cameras, *J. Constr. Eng. Manag.* 136 (6) (2010) 632–640, [http://dx.doi.org/10.1061/\(ASCE\)JCO.1943-7862.0000164](http://dx.doi.org/10.1061/(ASCE)JCO.1943-7862.0000164).
- [14] T. Chen, N. Yabuki, T. Fukuda, Motion recognition method for construction workers using selective depth inspection and optimal inertial measurement unit sensors, *CivilEng* 4 (1) (2023) 204–223, <http://dx.doi.org/10.3390/civileng4010013>.
- [15] B. McCabe, H. Hamledari, A. Shahi, P. Zangeneh, E. Azar, Roles, benefits, and challenges of using uavs for indoor smart construction applications, 2017, pp. 349–357, <http://dx.doi.org/10.1061/9780784480830.043>.
- [16] I. Muhammad, K. Ying, M. Nithish, J. Xin, Z. Xinge, C.C. Cheah, Robot-assisted object detection for construction automation: Data and information-driven approach, *IEEE/ASME Trans. Mechatronics* 26 (6) (2021) 2845–2856, <http://dx.doi.org/10.1109/TMECH.2021.3100306>.
- [17] M.S. Moura, C. Rizzo, D. Serrano, BIM-based localization and mapping for mobile robots in construction, in: *2021 IEEE International Conference on Autonomous Robot Systems and Competitions, ICARSC, IEEE, Santa Maria da Feira, Portugal*, 2021, pp. 12–18, <http://dx.doi.org/10.1109/ICARSC52212.2021.9429779>.
- [18] A. Giusti, V. Magnago, D. Siegle, M. Terzer, C. Follini, S. Garbin, C. Marcher, D. Steiner, A. Schweigkofler, M. Riedl, BALTO: a BIM-integrated mobile robot manipulator for precise and autonomous disinfection in buildings against COVID-19, in: *2021 IEEE 17th International Conference on Automation Science and Engineering, CASE, IEEE, Lyon, France*, 2021, pp. 1730–1737, <http://dx.doi.org/10.1109/CASE49439.2021.9551635>.
- [19] X. Zhao, C.C. Cheah, BIM-based indoor mobile robot initialization for construction automation using object detection, *Autom. Constr.* 146 (2023) 104647, <http://dx.doi.org/10.1016/j.autcon.2022.104647>.
- [20] R.W.M. Hendriks, P. Pauwels, E. Torta, H.J. Bruyninckx, M.J.G. Van De Molengraft, Connecting semantic building information models and robotics: An application to 2D LiDAR-based localization, in: *2021 IEEE International Conference on Robotics and Automation, ICRA, IEEE, Xi'an, China*, 2021, pp. 11654–11660, <http://dx.doi.org/10.1109/ICRA48506.2021.9561129>.
- [21] H. Yin, Z. Lin, J.K.W. Yeoh, Semantic localization on BIM-generated maps using a 3D LiDAR sensor, *Autom. Constr.* 146 (2023) 104641, <http://dx.doi.org/10.1016/j.autcon.2022.104641>.

- [22] A. Spinner, A. Degani, Online as-built building information model update for robotic monitoring in construction sites, *J. Intell. Robot. Syst.* 110 (2) (2024) 50, <http://dx.doi.org/10.1007/s10846-024-02087-2>.
- [23] S. Karimi, R.G. Braga, I. Iordanova, D. St-Onge, Semantic navigation using building information on construction sites, 2021, <http://dx.doi.org/10.48550/arXiv.2104.10296>, arXiv:2104.10296.
- [24] S. Karimi, R.G. Braga, I. Iordanova, D. St-Onge, Semantic optimal robot navigation using building information on construction sites, in: International Symposium on Automation and Robotics in Construction (ISARC) Proceedings 2021 Proceedings of the 38th ISARC, Dubai, UAE, IAARC, 2021, pp. 57–64, <http://dx.doi.org/10.22260/ISARC2021/0010>.
- [25] M.A. Gopee, S.A. Prieto, B. García De Soto, IFC-based generation of semantic obstacle maps for autonomous robotic systems, 2022, <http://dx.doi.org/10.35490/EC3.2022.161>.
- [26] P. Pauwels, R. de Koning, B. Hendrikx, E. Torta, Live semantic data from building digital twins for robot navigation: Overview of data transfer methods, *Adv. Eng. Inform.* 56 (2023) 101959, <http://dx.doi.org/10.1016/j.aei.2023.101959>.
- [27] S. Park, S. Yoon, S. Ju, J. Heo, BIM-based scan planning for scanning with a quadruped walking robot, *Autom. Constr.* 152 (2023) 104911, <http://dx.doi.org/10.1016/j.autcon.2023.104911>.
- [28] R. Zhai, J. Zou, V.J.L. Gan, X. Han, Y. Wang, Y. Zhao, Semantic enrichment of BIM with IndoorGML for quadruped robot navigation and automated 3D scanning, *Autom. Constr.* 166 (2024) 105605, <http://dx.doi.org/10.1016/j.autcon.2024.105605>.
- [29] H. Liao, B. Huang, K. Dong, Z. Jia, J. Wu, Robot-assisted after-process progress-monitoring system based on BIM and computer vision, in: 2023 IEEE International Conference on Robotics and Biomimetics, ROBIO, 2023, pp. 1–6, <http://dx.doi.org/10.1109/ROBIO58561.2023.10355015>.
- [30] K.S. Saidi, A.M. Lytle, W.C. Stone, Report of the NIST workshop on data exchange standards at the construction job site, in: International Symposium on Automation and Robotics in Construction (ISARC) Proceedings 2003 Proceedings of the 20th ISARC, Eindhoven, Holland, IAARC, 2003, pp. 617–622, <http://dx.doi.org/10.22260/ISARC2003/0095>.
- [31] I.N. Davidson, M.J. Skibniewski, Simulation of automated data collection in buildings, *J. Comput. Civ. Eng.* 9 (1) (1995) 9–20, [http://dx.doi.org/10.1061/\(ASCE\)0887-3801\(1995\)9:1\(9\)](http://dx.doi.org/10.1061/(ASCE)0887-3801(1995)9:1(9)).
- [32] R. Navon, R. Sacks, Assessing research issues in automated project performance control (APPC), *Autom. Constr.* 16 (4) (2007) 474–484, <http://dx.doi.org/10.1016/j.autcon.2006.08.001>.
- [33] A. Costin, N. Pradhananga, J. Teizer, Leveraging passive RFID technology for construction resource field mobility and status monitoring in a high-rise renovation project, *Autom. Constr.* 24 (2012) 1–15, <http://dx.doi.org/10.1016/j.autcon.2012.02.015>.
- [34] E. Ergen, B. Akinci, R. Sacks, Tracking and locating components in a precast storage yard utilizing radio frequency identification technology and GPS, *Autom. Constr.* 16 (3) (2007) 354–367, <http://dx.doi.org/10.1016/j.autcon.2006.07.004>.
- [35] H. Bardareh, O. Moselhi, An integrated RFID-UWB method for indoor localization of materials in construction, *J. Inf. Technol. Constr.* 27 (2022) 642–661, <http://dx.doi.org/10.36680/j.itcon.2022.032>.
- [36] F. Bosché, Automated recognition of 3D CAD model objects in laser scans and calculation of as-built dimensions for dimensional compliance control in construction, *Informatics for Cognitive Robots, Adv. Eng. Inform.* 24 (1) (2010) 107–118, <http://dx.doi.org/10.1016/j.aei.2009.08.006>.
- [37] M. Golparvar-Fard, J. Bohn, J. Teizer, S. Savarese, F. Peña-Mora, Evaluation of image-based modeling and laser scanning accuracy for emerging automated performance monitoring techniques, *Autom. Constr.* 20 (8) (2011) 1143–1155, <http://dx.doi.org/10.1016/j.autcon.2011.04.016>.
- [38] A. Shahi, J.M. Cardona, C.T. Haas, J.S. West, G.L. Caldwell, Activity-based data fusion for automated progress tracking of construction projects, 2012, pp. 838–847, <http://dx.doi.org/10.1061/9787084412329.085>.
- [39] A.S. Barbosa, D.B. Costa, Use of BIM and visual data collected by UAS and 360° camera for construction progress monitoring, *IOP Conf. Ser.: Earth Environ. Sci.* 1101 (8) (2022) 082007, <http://dx.doi.org/10.1088/1755-1315/1101/8/082007>.
- [40] R. Navon, Research in automated measurement of project performance indicators, *Autom. Constr.* 16 (2) (2007) 176–188, <http://dx.doi.org/10.1016/j.autcon.2006.03.003>.
- [41] T. Meyer, A. Brunn, U. Stilla, Change detection for indoor construction progress monitoring based on BIM, point clouds and uncertainties, *Autom. Constr.* 141 (2022) 104442, <http://dx.doi.org/10.1016/j.autcon.2022.104442>.
- [42] A. Ibrahim, A. Sabet, M. Golparvar-Fard, BIM-driven mission planning and navigation for automatic indoor construction progress detection using robotic ground platform, 2019, pp. 182–189, <http://dx.doi.org/10.35490/EC3.2019.195>.
- [43] M.A.V. Torres, A. Braun, A. Borrmann, Occupancy grid map to pose graph-based map: Robust BIM-based 2D-LiDAR localization for lifelong indoor navigation in changing and dynamic environments, 2023, pp. 567–574, <http://dx.doi.org/10.1201/9781003354222-72>, arXiv:2308.05443.
- [44] M.A.V. Torres, A. Braun, A. Borrmann, BIM-SLAM: integrating BIM models in multi-session SLAM for lifelong mapping using 3D LiDAR, 2023, <http://dx.doi.org/10.22260/ISARC2023/0070>, arXiv:2408.15870.
- [45] L. Schaub, I. Podkosova, C. Schonauer, H. Kaufmann, Point cloud to BIM registration for robot localization and augmented reality, in: 2022 IEEE International Symposium on Mixed and Augmented Reality Adjunct, ISMAR-Adjunct, IEEE, Singapore, Singapore, 2022, pp. 77–84, <http://dx.doi.org/10.1109/ISMAR-Adjunct57072.2022.00025>.
- [46] H. Yin, J.M. Liew, W.L. Lee, M.H. Ang, Ker-Wei Yeoh, Justin, Towards BIM-based robot localization: A real-world case study, 2022, <http://dx.doi.org/10.22260/ISARC2022/0012>.
- [47] S. Shan, D. Xu, J. Gong, Real time localization method research with monoslam and BIM model, *IOP Conf. Ser.: Earth Environ. Sci.* 768 (1) (2021) 012126, <http://dx.doi.org/10.1088/1755-1315/768/1/012126>.
- [48] M. Giberna, M. Shaheer, H. Bavl, J.A. Millan-Romera, J.L. Sanchez-Lopez, H. Voos, Constraint-based modeling of dynamic entities in 3D scene graphs for robust SLAM, 2025, <http://dx.doi.org/10.48550/arXiv.2503.02050>, arXiv:2503.02050.
- [49] P. Racinskis, J. Arents, M. Greitans, Constructing maps for autonomous robotics: An introductory conceptual overview, *Electronics* 12 (13) (2023) 2925, <http://dx.doi.org/10.3390/electronics12132925>.
- [50] A. Elfes, Occupancy grids: A stochastic spatial representation for active robot perception, 2013, arXiv:1304.1098.
- [51] M.B. Alatise, G.P. Hancke, A review on challenges of autonomous mobile robot and sensor fusion methods, *IEEE Access* 8 (2020) 39830–39846, <http://dx.doi.org/10.1109/ACCESS.2020.2975643>.
- [52] X. Hu, R.H. Assaad, A BIM-enabled digital twin framework for real-time indoor environment monitoring and visualization by integrating autonomous robotics, LiDAR-based 3D mobile mapping, IoT sensing, and indoor positioning technologies, *J. Build. Eng.* 86 (2024) 108901, <http://dx.doi.org/10.1016/j.jobeb.2024.108901>.
- [53] M.A. Gopee, S.A. Prieto, B. García De Soto, Improving autonomous robotic navigation using IFC files, *Constr. Robot.* 7 (3–4) (2023) 235–251, <http://dx.doi.org/10.1007/s41693-023-00112-8>.
- [54] W. Palacz, G. Ślusarczyk, B. Strug, E. Grabska, Indoor robot navigation using graph models based on BIM/IFC, in: L. Rutkowski, R. Scherer, M. Korytkowski, W. Pedrycz, R. Tadeusiewicz, J.M. Zurada (Eds.), in: Artificial Intelligence and Soft Computing, vol. 11509, Springer International Publishing, Cham, 2019, pp. 654–665, http://dx.doi.org/10.1007/978-3-030-20915-5_58.
- [55] A. Hamieh, A. Ben Makhlof, B. Louhichi, D. Deneux, A BIM-based method to plan indoor paths, *Autom. Constr.* 113 (2020) 103120, <http://dx.doi.org/10.1016/j.autcon.2020.103120>.
- [56] X. Liu, C. He, H. Zhao, J. Jia, C. Liu, Building information modeling indoor path planning: A lightweight approach for complex BIM building, *Comput. Animat. Virtual Worlds* 32 (3–4) (2021) e2014, <http://dx.doi.org/10.1002/cav.2014>.
- [57] R. Monica, J. Aleotti, Surfel-based next best view planning, *IEEE Robot. Autom. Lett.* 3 (4) (2018) 3324–3331, <http://dx.doi.org/10.1109/LRA.2018.2852778>.
- [58] R. Zeng, Y. Wen, W. Zhao, Y.-J. Liu, View planning in robot active vision: A survey of systems, algorithms, and applications, *Comput. Vis. Media* 6 (3) (2020) 225–245, <http://dx.doi.org/10.1007/s41095-020-0179-3>.
- [59] A.K. Burusa, E.J. van Henten, G. Kootstra, Gradient-based local next-best-view planning for improved perception of targeted plant nodes, 2024, <http://dx.doi.org/10.48550/arXiv.2311.16759>, arXiv:2311.16759.
- [60] A. Braun, S. Tutas, A. Borrmann, U. Stilla, Improving progress monitoring by fusing point clouds, semantic data and computer vision, *Autom. Constr.* 116 (2020) 103210, <http://dx.doi.org/10.1016/j.autcon.2020.103210>.
- [61] A. Pal, J.J. Lin, S.-H. Hsieh, M. Golparvar-Fard, Activity-level construction progress monitoring through semantic segmentation of 3D-informed orthographic images, *Autom. Constr.* 157 (2024) 105157, <http://dx.doi.org/10.1016/j.autcon.2023.105157>.
- [62] B. Ekanayake, A. Ahmadian Fard Fini, J.K.W. Wong, P. Smith, A deep learning-based approach to facilitate the as-built state recognition of indoor construction works, *Constr. Innov.* 24 (4) (2024) 933–949, <http://dx.doi.org/10.1108/CI-05-2022-0121>.
- [63] O. Saha, G.V. Horn, S. Maji, Improved zero-shot classification by adapting VLMs with text descriptions, in: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, IEEE Computer Society, 2024, pp. 17542–17552, <http://dx.doi.org/10.1109/CVPR52733.2024.01661>.
- [64] M. Yi, Z. Xie, Y. Yang, C. Liu, Y. Ding, H. Lu, Decoupling classification and localization of CLIP, in: 2024 IEEE International Conference on Multimedia and Expo Workshops, ICMEW, 2024, pp. 1–6, <http://dx.doi.org/10.1109/ICMEW63481.2024.10645474>.
- [65] M.-Y. Cheng, J.-C. Chen, Integrating barcode and GIS for monitoring construction progress, *Autom. Constr.* 11 (1) (2002) 23–33, [http://dx.doi.org/10.1016/S0926-5805\(01\)00043-7](http://dx.doi.org/10.1016/S0926-5805(01)00043-7).
- [66] J. Liu, S. Ergen, Q. Zhang, An AprilTags-based approach for progress monitoring and quality control in modular construction, *Buildings* 14 (7) (2024) 2252, <http://dx.doi.org/10.3390/buildings14072252>.
- [67] M. Golparvar-Fard, F. Peña-Mora, S. Savarese, Monitoring changes of 3D building elements from unordered photo collections, in: 2011 IEEE International Conference on Computer Vision Workshops, ICCV Workshops, 2011, pp. 249–256, <http://dx.doi.org/10.1109/ICCVW.2011.6130250>.

- [68] A. Hornung, K.M. Wurm, M. Bennewitz, C. Stachniss, W. Burgard, OctoMap: an efficient probabilistic 3D mapping framework based on octrees, *Auton. Robots* (2013) <http://dx.doi.org/10.1007/s10514-012-9321-0>.
- [69] S. Macenski, M. Booker, J. Wallace, Open-source, cost-aware kinematically feasible planning for mobile and surface robotics, 2024, <http://dx.doi.org/10.48550/arXiv.2401.13078>, [arXiv:2401.13078](https://arxiv.org/abs/2401.13078).
- [70] Y. Zheng, A. Al Barazi, O. Seppänen, H. Abou-Ibrahim, C. Görsch, Semantic digital twin framework for monitoring construction workflows, *Autom. Constr.* 176 (2025) 106301, <http://dx.doi.org/10.1016/j.autcon.2025.106301>, URL <https://www.sciencedirect.com/science/article/pii/S0926580525003413>.